

The 2025 Google Code Golf Championship

Michael D. Moffitt

MOFFITT@GOOGLE.COM

Divy Thakkar

DTHAKKAR@GOOGLE.COM

Ryan Burnell

RBURNELL@GOOGLE.COM

Orhan Firat

ORHANF@GOOGLE.COM

Walter Reade

INVERSION@GOOGLE.COM

Sohier Dane

SOHIER@GOOGLE.COM

Addison Howard

ADDISONHOWARD@GOOGLE.COM

Competition Organizers

Lucas Deutschmann

LUCAS.DEUTSCHMANN@RPTU.DE

Luke Gustafson

LUKEG3@GMAIL.COM

Adam Kues

AK@AKORLITH.COM

Mukundan Senthilkumar

MUKUNDAN314@GMAIL.COM

Team: Code Golf International (1st Place)

Nicolas Posada

POSADANICO@OUTLOOK.COM

Lyndon Ho

HLYNDON20@GMAIL.COM

Larry Wu

WUPIR2@GMAIL.COM

Lav Patel

SUSPICIOUS.PMLR.EMAIL@GMAIL.COM

Team: jailctf merger (2nd Place)

Hunter Richardson

HUNTERTR2002@GMAIL.COM

Wei Heng Kuan

KUANWH1996@GMAIL.COM

Team: ox jam! (3rd Place)

Editors: Tao Qin, Kun Zhang, and Jes Frellsen

Abstract

For over half a decade, the Abstraction and Reasoning Corpus has played a critical role in evaluating the fluid intelligence and skill acquisition efficiency of machine learning systems. While the dataset has proven especially useful in advancing program synthesis, few investments have been made in distilling the transformational logic of these tasks into their most irreducible computational forms. In conjunction with the NeurIPS 2025 Competition Track, we organized the *2025 Google Code Golf Championship*, a three-month online challenge designed to elicit minimal Python solutions for all training puzzles in the seminal ARC-AGI-1 benchmark suite. Sponsored by Google DeepMind and funded with \$100,000 in prizes, the contest attracted *over 1,000 teams* and yielded tens of thousands of verified, open-source programs that collectively establish tight upper bounds on the Kolmogorov complexity of each task. In this paper, we offer a comprehensive retrospective of the event, including an overview of the contest format, a survey of approaches taken by top teams, and a discussion of potential implications for the future design of intelligent agents.

1. Introduction

The Abstraction and Reasoning Corpus (ARC-AGI) has established itself as a cornerstone benchmark for measuring fluid intelligence and generalization in machine learning systems (Chollet, 2019). Unlike traditional datasets that reward pattern matching via manifold interpolation, ARC-AGI is a few-shot problem-solving suite designed to test an agent’s ability to efficiently acquire new skills when presented with novel, previously unseen tasks. By focusing on core knowledge priors — e.g., translation, symmetry, counting, etc. — rather than semantic knowledge, the benchmark provides a unique litmus test for evaluating progress toward achieving general intelligence. Its rigor lies in its refusal to allow memorization, as a successful system cannot merely recall data, but must instead actively reason about the underlying generative process demonstrated in each puzzle.

Among the most promising approaches to solving ARC is the application of deep learning-guided program synthesis, where neural networks direct the search for symbolic programs that satisfy the task examples (Alford et al., 2022; Ainooson et al., 2023; Neumann and Pintér, 2023; Xu et al., 2023; Bober-Irizar and Banerjee, 2024; Lei et al., 2024; Ouellette, 2024; Akyürek et al., 2025; Hocquette and Cropper, 2025; Li et al., 2025; Liao and Gu, 2025; Macfarlane and Bonnet, 2025; Pourcel et al., 2025; Rocha et al., 2025; Witt et al., 2025). Yet, compared to conventional software engineering patterns (such as those found in massive pre-training corpora like GitHub), the puzzles embodied by ARC represent a significant distributional shift, with the visual-spatial logic required to manipulate grids having relatively little overlap with standard coding benchmarks. Compounding this challenge is the fact that each transformation in ARC is defined only implicitly as input-output image pairs: given that no canonical set of source codes is bundled with the benchmark suite to serve as ground truth, this absence represents a critical gap for researchers. Without access to verified, human-authored logic adapted to this domain, it remains difficult to effectively fine-tune agents or to evaluate the efficiency of the programs they generate (Chen et al., 2021; Du et al., 2024; Luo et al., 2024; Wei et al., 2024; Zhu et al., 2024; Qiu et al., 2025).

To bridge this gap, we adopted a competitive incentive structure similar in spirit to the ARC Prize (Chollet et al., 2024, 2026) by organizing the *2025 Google Code Golf Championship* in conjunction with the NeurIPS 2025 Competition Track. Specifically, we framed the creation of reference implementations as an instance of *code golf* — a decades-old recreational programming pursuit in which participants strive to solve a particular problem using the absolute minimum number of bytes or characters (see Figure 1 for two other examples). This format naturally aligns with the theoretical principle of Minimum Description Length (MDL), effectively leveraging the optimization pressure of the game to establish tight upper bounds on the Kolmogorov complexity of each task (Solomonoff, 1964; Kolmogorov, 1965; Chaitin, 1966; Rissanen, 1978; Barron et al., 1998; Grünwald, 2004; Li and Vitányi, 2019). Crucially, our competition remained agnostic regarding the nature of the participating intelligence: we invited solutions derived from purely human intuition, automated program synthesis, or hybrid human-AI workflows. These open parameters elicited exceptional algorithmic creativity, requiring teams to identify and exploit structural regularities to shave bytes from their submissions. By channeling this collective ingenuity, the championship succeeded in crowdsourcing highly compressed Python solutions for the entire ARC-AGI-1 training set.

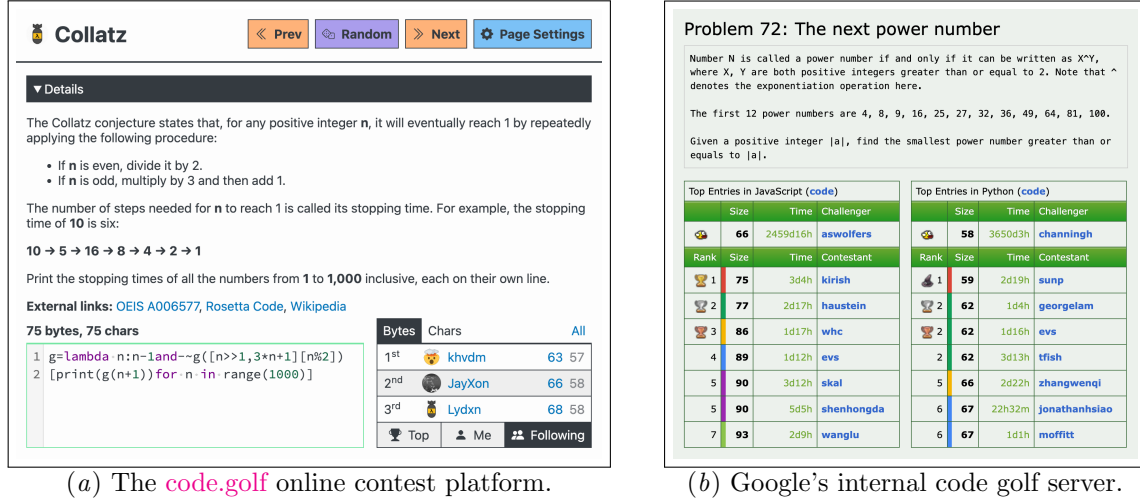


Figure 1: Screenshots illustrating the long-standing culture of competitive code minimization, spanning (a) the well-established `code.golf` public platform active since 2017, and (b) an internal server used by Google engineers since 2014.

The result is a threefold contribution to the scientific community:

- A novel set of minimal reference programs from winning teams that faithfully approximate the true complexity of the underlying task mechanics.
- A massive corpus of verified solutions produced by dozens of other participating teams that, while not necessarily minimal, nevertheless represent an earnest attempt at optimizing lexical density, capturing a wide variety of algorithmic approaches.
- A rich repository of community-generated resources, including open-source validation tools and qualitative “golfer’s guides” that explicitly codify the strategies used to compress the abstract priors of ARC-AGI into minimal Python logic.

2. Contest Format

Our contest was administered by `Kaggle`, an online community and competition platform for machine learning practitioners and data scientists. The site allows users to collaborate, compete, and access a vast library of public datasets, code, and tutorials. Over the past decade, it has hosted hundreds of high-profile competitions (including the original Abstraction and Reasoning Challenge, along with both the 2024 and 2025 editions of the ARC Prize)^{1,2,3} and has become a crucial hub for developing skills, networking, and advancing the field of machine learning.

1. <https://kaggle.com/competitions/abstraction-and-reasoning-challenge>
2. <https://kaggle.com/competitions/arc-prize-2024>
3. <https://kaggle.com/competitions/arc-prize-2025>

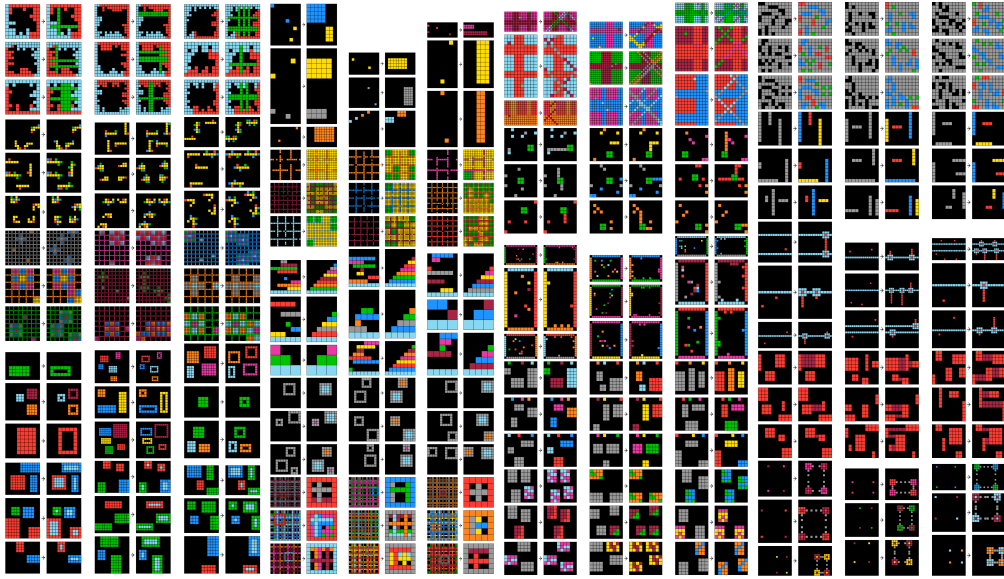


Figure 2: A subset of input/output pairs taken from the ARC-GEN-100K dataset.

2.1. Evaluation

For each task in the ARC-AGI-1 training benchmark suite, each team earned a score of $\max(1, 2500 - \text{length})$ points for a functionally correct Python program using *length* characters (or more specifically, bytes) in its definition. Any functionally incorrect program — i.e., that produced an inconsistent outcome for some $\langle \text{input}, \text{output} \rangle$ pair — earned 0.001 points, allowing teams to deduce the overall number of failures from their total score alone. We believe this scoring system offered several key advantages:

- It was extremely *easy* for both participants and contest organizers to measure.
- It was entirely *objective*, leaving very little room for error or misinterpretation.
- It encouraged *short* programs likely to be most useful in establishing a canonical reference suite.

2.2. Datasets

To establish the functional correctness of a submission, we employed two separate validation datasets. The first is the set of demonstration exemplars included in the official ARC-AGI-1 release, which contains between three and five image pairs for the majority of tasks.⁴ To disincentivize the creation of hardcoded solutions that overfit to these specific input/output grids, we incorporated elements from a second suite containing 100,000 additional pairs produced by ARC-GEN, a mimetic procedural benchmark generator designed to mirror the original distribution of examples as faithfully as possible (Moffitt, 2025).^{5,6} Refer to Figure 2 for a visual gallery of several such instances.

4. <https://github.com/fchollet/ARC-AGI>

5. <https://github.com/google/ARC-GEN>

6. <https://kaggle.com/datasets/arcgen100k/the-arc-gen-100k-dataset>

This starter notebook for the [2025 Google Code Golf Championship](#) is designed to help contestants verify the functional correctness of their programs. It allows one to load example pairs for a task, visualize them, and test whether a candidate code snippet produces expected results across all competition benchmarks.

Enter a task number (between 1 and 400):

```
task_num = 0 # Task 0 is just an illustrative example (and not eligible for points)
```

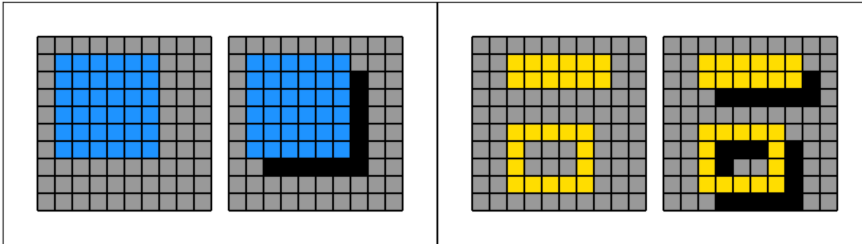
Color legend:

```
import sys
sys.path.append("/kaggle/input/google-code-golf-2025/code_golf_utils")
from code_golf_utils import *
show_legend()
```



Example <input, output> pairs:

```
examples = load_examples(task_num)
show_examples(examples['train'] + examples['test'])
```



Enter your code to solve the task:

```
%%writefile task.py
def p(g):
    for r, row in enumerate(g):
        for c, color in enumerate(row):
            if r and c and color==5 and g[r-1][c-1] not in [0,5]: g[r][c]=0
    return g
```

Writing task.py

Verify your code:

```
verify_program(task_num, examples)
```

Results on ARC-AGI examples: 2 pass, 0 fail

Results on ARC-GEN examples: 1 pass, 0 fail

Your code IS READY for submission!

Its length appears to be 150 bytes.

Next steps:

- * Copy it into a file named task000.py on your local machine.
- * Create a zip file containing that program along with all others.
- * Submit that zip file to the Kaggle competition so that it can be officially scored.

Figure 3: The official competition starter notebook.

Notebook Title	Author	Score	Link
A bit more of Code Golf: 255/400 + visualization	jacekw1	578,523.147	[link]
Qwen2.5 32B + ARC + Local Score	bibanh	640,225.123	[link]
Code Golf - Ensemble - Local Score - 391/400 DSL	seshurajup	677,725.013	[link]
Oh Barnacles	jazivxt	737,616.018	[link]
Road to 400 collaboration	mbilichenko	819,231.002	[link]
python-minifier applied	vladislavlassa	851,678.000	[link]
Big Zippa	cheeseexports	877,446.000	[link]
Solutions for all 400 Tasks + Ensemble	kerta27	899,523.000	[link]
GCGC playground	jonathanchan	912,484.000	[link]
Google Code Golf Championship 101	adilshamim8	913,211.000	[link]
Google Code Golf Community Best	tonyllica	913,449.000	[link]
<i>Compilation of winning solutions*</i>	garrymoss	963,258.000	[link]

*post-competition notebook

Table 1: Notable notebooks.

2.3. Timeline

- June 27th, 2025 — Announcement of accepted NeurIPS 2025 Competitions.⁷
- June 27th, 2025 — Contest pre-registration begins.⁸
- July 31st, 2025 — Start date and launch of the Kaggle site.⁹
- October 23rd, 2025 — Entry deadline & team merger deadline.
- October 30th, 2025 — Final submission deadline.

3. Notable Notebooks

Contestants were supplied with a starter notebook (see Figure 3) designed to assist with the construction and verification of programs.¹⁰ This interactive script allowed participants to load examples for a task, visualize them, and develop the corresponding Python solution. Teams would then create a zip file containing all such programs and submit this for official scoring and placement on a live leaderboard (see Appendix A). Throughout the competition, users also had an opportunity to create and share their own notebooks, resulting in **over 140 public notebooks** of which twelve are highlighted in Table 1. Several tricks and techniques revealed in these codes made their way into the submissions of top-scoring teams.

7. <https://blog.neurips.cc/2025/06/27/neurips-2025-competitions-announced/>
8. <https://sites.google.com/corp/view/neurips-2025-code-golf>
9. <https://kaggle.com/competitions/google-code-golf-2025>
10. <https://kaggle.com/code/mhoffitt/neurips-2025-google-code-golf-championship>

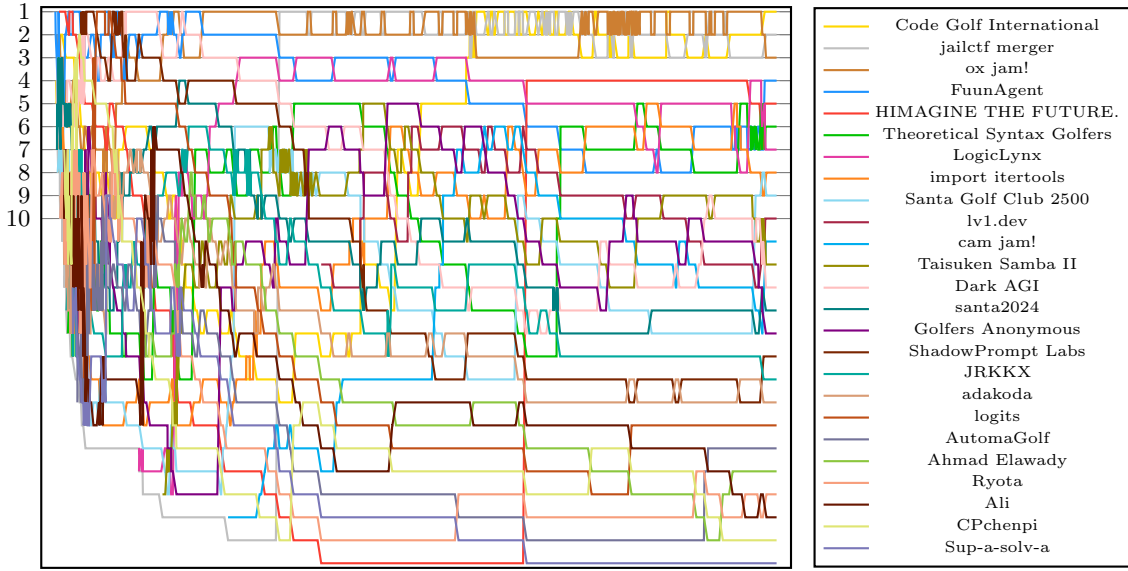


Figure 4: A time series of team ranks.

4. Results

Our competition attracted a total of **6,306** entrants, yielding **41,314** submissions from **1,142 teams**. In Figure 4, we present a time series of ranks across all teams who placed within the top ten at any point following the first week of the contest. At the end of the competition, the highest-ranking team had earned a total of 962,070 points, while the aggregate “super-score” — calculated by taking the best solutions discovered among all participating teams — reached an astonishing **963,226** points, translating to an average solution length of approximately **92 bytes per task** (less than half that of any public notebook). Complete statistics and codes from the shortest programs are provided in Appendix D.

5. Team Strategies

Full descriptions of the strategies employed by the 1st, 2nd, and 3rd place teams are included in Appendix B. Given that ARC-AGI was explicitly designed to resist conventionally-trained AI systems, it may come as little surprise that no fully-automated end-to-end pipeline emerged from the contest. Yet — perhaps remarkably — none of these top three teams employed *any* form of Artificial Intelligence in the production of their codes, instead relying exclusively upon (a) their personal expertise in manual golfing, and (b) a combination of bespoke utilities for code minimization. As shown in Figure 5, this helped place considerable distance between their total byte counts and those of other winning teams (whose approaches are summarized in the sections below). In Table 2, we provide a link to each team’s post-competition solution write-up, along with their final score and a distribution of their “golf medals”.¹¹

11. In code golf vernacular, a *{gold, silver, bronze}* medal is earned for every *{shortest, second-shortest, third-shortest}* solution (respectively), and a *diamond* if no other team matched the shortest solution.

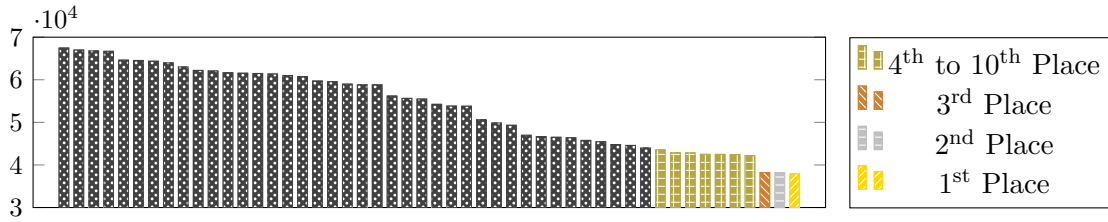


Figure 5: Distribution of total byte counts.

5.1. The “Human-in-the-Loop” vs. Manual Implementation Spectrum

Teams varied significantly in their reliance on Artificial Intelligence, ranging from nearly full automation to purely manual golfing.

- **LLM-Driven Automation:** The 4th place team used a “human-in-the-loop” algorithm where 98% of their solutions were generated by an LLM (specifically gpt5-codex). They ran parallel sampling sessions ($N = 4$) and used rule-based prompts generated from the code’s Abstract Syntax Tree (AST) to suggest specific improvements, such as converting loops to recursion.
- **Semi-Automation:** The 7th place team used a semi-automated workflow with the Codex web interface, selecting the shortest of 4 generated outputs. They provided XML-structured prompts and specific “work history” contexts to prevent the AI from repeating failed experiments.
- **DSL Experimentation:** The 5th place team initially experimented with a domain-specific language previously developed for the original ARC challenge (Hodel, 2024), achieving compressed sizes around 2000 bytes.¹² However, they ultimately abandoned this approach in favor of handwritten solutions to achieve better results.
- **Manual Golfing:** Similarly, the 8th and 9th place teams found LLMs insufficient for top-tier scores and pivoted to manual implementation. The 9th place team spent over 1,000 hours manually optimizing code, using LLMs only for learning concepts rather than generating solutions.

5.2. Advanced Compression Pipelines (Zopfli & Custom Tools)

Standard libraries were often insufficient; teams built sophisticated compression pipelines to shave off bytes.

- **Zopfli over Zlib:** Multiple teams (5th, 7th, 8th) identified `zopfli` as superior to the standard Python `zlib` library for compression ratios.
- **Variable Name Optimization:** Teams discovered that variable naming impacts Huffman coding efficiency in compression. The 5th and 6th place teams used static analysis and local search (e.g., genetic algorithms or simulated annealing) to rename

12. See <https://github.com/michaelhodel/arc-dsl> and <https://github.com/michaelhodel/re-arc>.

Team Rank	Team Name	Final Score	# Golf Medals				Solution Write-Up
			Diamond	Gold	Silver	Bronze	
#1	Code Golf International	962,070	80	229	80	43	[link]
#2	jailctf merger	961,805	55	192	94	56	[link]
#3	ox jam!	961,784	70	194	80	50	[link]
#4	FuunAgent	957,810	3	74	23	13	[link]
#5	HIMAGINE THE FUTURE.	957,568	0	58	14	10	[link]
#6	Theoretical Syntax Golfers	957,527	5	63	22	18	[link]
#7	LogicLynx	957,513	8	89	29	21	[link]
#8	import itertools	957,177	1	74	11	6	[link]
#9	Santa Golf Club 2500	957,151	0	70	13	9	[link]
#10	lv1.dev	956,458	3	95	22	15	[link]

Table 2: The winning teams.

variables to be “compression-friendly.” Similarly, the 7th and 8th place teams ran variable name searches to optimize compressibility.

- **Latin-1 Encoding:** To avoid the overhead of multibyte Unicode characters in Python strings, teams encoded compressed payloads using Latin-1 (`#coding: L1`), which represents all 256 of its characters with one byte.
- **Custom Decompression Stubs:** Teams optimized the decompression code itself, often using `exec(zlib.decompress(..., -9))` to handle raw deflate streams and minimize header overhead.

5.3. Exploiting Domain Specifics: Seed Cracking

Some teams bypassed the logic puzzles entirely by analyzing the test case generation engine.

- **Recovering Random Seeds:** The 5th place team analyzed the ARC-GEN generator and discovered that `random.seed` was often predictable based on task parameters like grid size and color count.
- **Regeneration:** By recovering the seed, they could regenerate the exact solution grid using a short random generator script rather than solving the puzzle logic, significantly compressing the solution size.

5.4. Common Python Golfing Idioms

Several specific syntactic patterns appeared repeatedly across different teams’ solutions.

- **Recursive Templates:** A dominant pattern for grid updates was using lambda recursion (e.g., `p=lambda g, i=count: -i*g or p(..., i-1)`) to repeat logic without explicit loops.

- **Rotation and Transposition:** Teams frequently used `[*zip(*g)]` for transposition and `[*zip(*g[::-1])]` for rotation. This allowed them to write logic for one direction and rotate the grid to apply it in all four directions.
- **Regular Expressions:** Regex was heavily utilized for pattern recognition and replacement via `re.sub` on the string representation of the grid. The 4th place team used AST analysis to prompt LLMs to try regex, while the 5th place team used advanced conditional group syntax.
- **exec Loops:** For complex iterations, using `exec` with string repetition was a common byte-saving alternative to standard loops (e.g., `exec("..."*count)`).

5.5. Infrastructure and Tooling

Success was supported by robust tooling built around the golfing process.

- **Dashboards:** Teams built custom web dashboards to track progress, visualize score trajectories, and identify tasks where they lagged behind best-known results.
- **CI/CD Integration:** The 4th place team implemented CI/CD pipelines to auto-generate README stats and notify the team via Discord upon every push, facilitating rapid iteration.
- **Search Tools:** Teams utilized tools like `pysearch` to brute-force short Python expressions for specific logic.

6. Conclusion

While our code golf championship was framed as a gamified test of software engineering ingenuity, the collective output of its participants has delivered far-reaching implications for the future of machine learning and agentic systems. What began as a recreational challenge for computer programming enthusiasts has culminated in a foundational contribution to the field of automated reasoning. The resulting repository — comprising tens of thousands of verified, highly compressed Python solutions — transforms what was once a collection of abstract visual puzzles into a rigorously defined corpus of logic. These open-source artifacts can now serve the critical scientific function of establishing precise complexity bounds for the ARC-AGI-1 benchmark suite and providing the ground truth necessary to train the next generation of coding agents. Ultimately, this initiative demonstrates that the path to solving general intelligence may not lie solely in massive scale, but also in the creative coordination of human intellect and expertise.

Acknowledgments

We are grateful to the NeurIPS 2025 Competition Track Chairs (specifically Jes Frellsen, Kun Zhang, and Tao Qin) for reviewing and accepting our proposal, and to the members of the ARC Prize Foundation (including François Chollet, Greg Kamradt, Mike Knoop, and Bryan Landers) for allowing us to use their benchmark suite as the basis for our challenge. Finally, we wish to thank Google leadership (particularly Samrat Ghosh, Ya Xu, and Zongwei Zhou) for lending their support to this event.

References

- James Ainooson, Deepayan Sanyal, Joel P. Michelson, Yuan Yang, and Maithilee Kunda. A Neurodiversity-Inspired Solver for the Abstraction & Reasoning Corpus (ARC) Using Visual Imagery and Program Synthesis, 2023. URL <https://arxiv.org/abs/2302.09425>.
- Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. The Surprising Effectiveness of Test-Time Training for Few-Shot Learning. In *Proceedings of the 42nd International Conference on Machine Learning*, ICML 2025, 2025. URL <https://openreview.net/forum?id=asgBo3FNdg>.
- Simon Alford, Anshula Gandhi, Akshay Rangamani, Andrzej Banburski, Tony Wang, Sylee Dandekar, John Chin, Tomaso Poggio, and Peter Chin. Neural-Guided, Bidirectional Program Search for Abstraction and Reasoning. In *Proceedings of the 10th International Conference on Complex Networks and Their Applications*, volume 1, pages 657–668, 2022.
- Andrew Barron, Jorma Rissanen, and Bin Yu. The Minimum Description Length Principle in Coding and Modeling. *IEEE Transactions on Information Theory*, 44(6):2743–2760, 1998.
- Mikel Bober-Irizar and Soumya Banerjee. Neural networks for abstraction and reasoning. *Scientific Reports*, 14, 2024. URL <https://doi.org/10.1038/s41598-024-73582-7>.
- Gregory J. Chaitin. On the Length of Programs for Computing Finite Binary Sequences. *J. ACM*, 13(4):547–569, October 1966. URL <https://doi.org/10.1145/321356.321363>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. Evaluating Large Language Models Trained on Code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- François Chollet. On the Measure of Intelligence, 2019. URL <https://arxiv.org/abs/1911.01547>.
- François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2024: Technical Report, 2024. URL <https://arxiv.org/abs/2412.04604>.
- François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2025: Technical Report, 2026. URL <https://arxiv.org/abs/2601.10904>.
- Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. Evaluating Large Language Models in Class-Level Code Generation. In *Proceedings of the 46th International Conference on Software Engineering*, ICSE 2024, 2024. URL <https://doi.org/10.1145/3597503.3639219>.
- Peter Grünwald. A Tutorial Introduction to the Minimum Description Length Principle, 2004. URL <https://arxiv.org/abs/math/0406077>.

- Céline Hocquette and Andrew Cropper. Relational Decomposition for Program Synthesis. In *Proceedings of the 34th International Joint Conference on Artificial Intelligence, IJCAI 2025*, pages 4526–4534, 2025. URL <https://doi.org/10.24963/ijcai.2025/504>.
- Michael Hodel. Addressing the Abstraction and Reasoning Corpus via Procedural Example Generation, 2024. URL <https://arxiv.org/abs/2404.07353>.
- Andrey Kolmogorov. Three Approaches to the Quantitative Definition of Information. *Problems of Information Transmission*, 1(1), 1965.
- Chao Lei, Nir Lipovetzky, and Krista A. Ehinger. Generalized Planning for the Abstraction and Reasoning Corpus. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence*, AAAI 2024, pages 20168–20175, 2024. URL <https://ojs.aaai.org/index.php/AAAI/article/view/29996>.
- Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer Publishing Company, Incorporated, 4th edition, 2019. ISBN 978-3-030-11297-4. URL <https://doi.org/10.1007/978-3-030-11298-1>.
- Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M. Dunn, Hao Tang, Wei-Long Zheng, Yewen Pu, and Kevin Ellis. Combining Induction and Transduction for Abstract Reasoning. In *Proceedings of the 13th International Conference on Learning Representations, ICLR 2025*, 2025. URL <https://openreview.net/forum?id=UmdotAAVDe>.
- Isaac Liao and Albert Gu. ARC-AGI Without Pretraining, 2025. URL <https://arxiv.org/abs/2512.06104>.
- Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. In *Proceedings of the 12th International Conference on Learning Representations, ICLR 2024*, 2024. URL <https://openreview.net/forum?id=UnUwSIgK5W>.
- Matthew Macfarlane and Clément Bonnet. Searching Latent Program Spaces. In *Proceedings of the 39th Conference on Neural Information Processing Systems, NeurIPS 2025*, 2025. URL <https://openreview.net/forum?id=CsXKGIqZtr>.
- Michael D. Moffitt. ARC-GEN: A Mimetic Procedural Benchmark Generator for the Abstraction and Reasoning Corpus, 2025. URL <https://arxiv.org/abs/2511.00162>.
- Norbert Neumann and Ádám Pintér. Solving ARC with non-procedural program induction. In *Proceedings of the 23rd International Symposium on Computational Intelligence and Informatics, CINTI 2023*, pages 271–276, 2023. URL <https://doi.org/10.1109/CINTI59972.2023.10382009>.
- Simon Ouellette. Towards Efficient Neurally-Guided Program Induction for ARC-AGI, 2024. URL <https://arxiv.org/abs/2411.17708>.

- Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI. In *Proceedings of the 42nd International Conference on Machine Learning*, ICML 2025, 2025. URL <https://openreview.net/forum?id=z4IG090qt2>.
- Ruizhong Qiu, Weiliang Will Zeng, James Ezick, Christopher Lott, and Hanghang Tong. How Efficient is LLM-Generated Code? A Rigorous & High-Standard Benchmark. In *Proceedings of the 13th International Conference on Learning Representations*, ICLR 2025, 2025. URL <https://openreview.net/forum?id=suz4utPr9Y>.
- J. Rissanen. Modeling By Shortest Data Description. *Automatica*, 14(5):465–471, 1978. URL <https://www.sciencedirect.com/science/article/pii/0005109878900055>.
- Filipe Marinho Rocha, Inês Dutra, Vítor Santos Costa, and Luís Paulo Reis. Program Synthesis Using Inductive Logic Programming for the Abstraction and Reasoning Corpus. *Intelligenza Artificiale*, 19(2):85–101, 2025. URL <https://journals.sagepub.com/doi/10.1177/17248035251363178>.
- R. J. Solomonoff. A Formal Theory of Inductive Inference. Part I. *Information and Control*, 7(1):1–22, 1964. URL <https://www.sciencedirect.com/science/article/pii/S0019995864902232>.
- Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Empowering Code Generation with OSS-Instruct. In *Proceedings of the 41st International Conference on Machine Learning*, ICML 2024, 2024. URL <https://openreview.net/forum?id=XUeo0Bid3x>.
- Jonas Witt, Sebastijan Dumančić, Tias Guns, and Claus-Christian Carbon. A Divide, Align, and Conquer Strategy for Program Synthesis. *Journal of Artificial Intelligence Research*, 82:1961–1997, March 2025. URL <https://doi.org/10.1613/jair.1.16847>.
- Yudong Xu, Elias B. Khalil, and Scott Sanner. Graphs, Constraints, and Search for the Abstraction and Reasoning Corpus. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence*, AAAI 2023, pages 4115–4122, 2023. URL <https://doi.org/10.1609/aaai.v37i4.25527>.
- Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, et al. DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence, 2024. URL <https://arxiv.org/abs/2406.11931>.

THE 2025 GOOGLE CODE GOLF CHAMPIONSHIP

Appendix A. The Official Competition Leaderboard

#	Team	Members	Score	Entries
1	Code Golf International	    	 962070.000	384
2	jailctf merger	    	 961805.000	169
3	ox jam! 🍷🔪🐮🐮🐮🐮🐮🐮	    	 961784.000	330
4	FuunAgent	  	 957810.000	573
5	HIMAGINE THE FUTURE.	   	 957568.000	51
6	Theoretical Syntax Golfers	    	 957527.000	38
7	LogicLynx	    	 957513.000	88
8	import itertools	   	 957177.000	569
9	Santa Golf Club 2500	    	 957151.000	290
10	lv1.dev	 	 956458.000	39
11	cam jam! 🎆🎆🎆🎆🎆🎆🎆🎆	    	 956008.000	181
12	タイスカンガ II	    	 955424.000	112
13	暗黒AGI	    	 955227.000	2278
14	ALE-Agent🐟	    	 954571.000	55
15	santa2024	    	 954246.000	3216
16	mystery+bono+Lucien	    	 953632.000	314
17	Golfers Anonymous		 953512.000	48
18	Team JYCDT	    	 953336.000	1219
19	Team	    	 953039.000	887
20	ShadowPrompt Labs	    	 950693.000	22
21	abdikmnrstvwxyz	  	 950146.000	44
22	JRKXX	    	 949384.000	531
23	needAJob	    	 946216.000	64
24	R.U.M.		 946190.000	16
25	Ravi Annaswamy		 945774.000	154

Appendix B. Detailed Team Strategies

In the sections that follow, we include detailed strategies for each of the top three teams. The first two are taken nearly verbatim from their online Kaggle solution write-ups, while the third summarizes a presentation delivered at our in-person Special Session held during NeurIPS on December 7th, 2025 in San Diego, California.^{13,14,15}

B.1. Code Golf International (1st Place)

First and foremost, we are very grateful to the organizers of the competition for such a unique opportunity to engage in competitive code golf with the broader Kaggle community. The last three months have been a thrilling experience for all of us, and finishing in first is a tremendous honor.

B.1.1. OVERVIEW

Solutions

Our code repository is available online.¹⁶ The final contest submission was from [this commit](#). To reproduce our entry, you can download the repo and simply run `python auto_zip.py`, which will create the `submission.zip` file.

Approach

Our team consisted of 5 experienced code golfers. Our approach to this competition was very manual, with all solutions authored by hand. The only important automation was zlib-based compression, which applied to 21 of the 400 tasks (more on that below).

Results

Our strong results in the competition can be mainly attributed to expertise in golfing and a large investment of time. To help understand the time investment, the average task might take a few hours to hand-optimize to a high level (including time of both the original author and people who reviewed), plus many tasks needed to be revisited as we discovered new or improved techniques. Multiplying by 400 tasks and it becomes quite an undertaking, even for a team of 5 splitting the work.

The competition was extremely close among the top 3 teams, and there is no particular technique or strategy we possessed that gave a decisive advantage over 2nd or 3rd. Indeed, reviewing others' solutions post-competition, we missed some useful techniques that the other teams had.

B.1.2. TECHNIQUES

Because all the problems are grid-based, there were many recurring techniques in our solutions. In this section, we outline some of the highlights, focusing on techniques that were unique to this contest rather than standard Python golf knowledge. However, this is in no way a complete list: every problem was hand-optimized and so there are many unique tricks.

13. <https://kaggle.com/competitions/google-code-golf-2025/writeups/cgi>

14. <https://kaggle.com/competitions/google-code-golf-2025/writeups/solution>

15. <https://neurips.cc/virtual/2025/competition/127722>

16. <https://github.com/Seek64/NeurIPS-Code-Golf-2025>

Short grid operations

Because these operations are quite short, they were essential building blocks of solutions.

Grid operations

- `zip(*g)` transposes the grid
- `g[::-1]` vertically mirrors the grid
- `zip(*g[::-1])` rotates 90 degrees
- `sum(g, [])` flattens the 2-d grid to a 1-d list. Note that in many cases `sum(g,g)` could be used to save a byte.
- `filter(any,g)` removes empty rows

Row operations

- `min`, `max`, `any`, `all`, and `sorted` built-in functions were used ubiquitously. For `min`, `max`, and `sorted`, the `key` argument was often utilized, e.g., to get the least/most common color.
- `{}.fromkeys(r)` will get the unique colors, sorted in order of first appearance. Alternatively, a dict comprehension could be used: `{v:0 for v in r}`
- `filter(int,r)` removes black cells

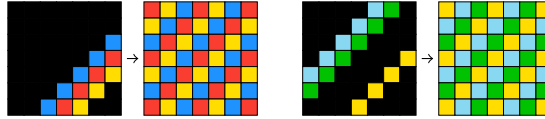
Recursion and loops

Another fundamental building block was using recursion or loops to repeat operations. Many solutions are of the form “do operation X, rotate 90 degrees, repeat 3 more times”.

The baseline recursion template is `p=lambda g,n=3:-n*g or F(p(g,n-1))`, taking advantage that `-n*g` is falsy for `n=0...3`. This is more efficient than writing out a loop such as `for n in range(4)`. In special cases, there are shorter methods or useful alternatives:

- If the recursion is just depth 2, then one can write `p=lambda g,h=0:F(h or p(g,g))`, saving 4 bytes in most cases. The variation `p=lambda g,*h:F(*h or p(*g))` saves another 2 bytes but removes the first row.
- Depth 2 recursion with transposition could sometimes be done as `p=lambda g:[*map(lambda r:...,*map(F,*g))]`, saving 1 more byte.
- If the grid is fixed height, then there’s a “slice recursion” technique, where you double the grid with each iteration and use a slice to stop the recursion when the desired depth is reached: `p=lambda g:F(g[70:] or p(g*2))` will iterate 4 times on a 10×10 grid. This saves bytes because it eliminates extra arguments to `p`. For example, [Task #093](#) uses `p=lambda g:g[57330:] or ...` to iterate 12 times over a 14×14 grid, because $57330 = 14 \times (2^{12} - 1)$.
- As an alternative to recursion, there’s a trick to make short loops: `p=lambda g:[g:=F(g) for _ in g][3]`. This is useful when `g` needs to be used multiple times inside `F`.

- `exec/eval` can be used with string multiplication to repeat code, e.g., `exec("F(x);"*9)`. It's very situational whether it's useful, but it can lead to small byte saves in a number of cases. An example is [Task #007](#):



```
p=lambda g:eval(f' [{"max(sum(g:=g[1:3]+g, []) [2::3]),"*7"}], '*7)
```

In this task, there is a pattern that repeats with length 3, and we must fill in the missing cells. To do so, we take the maximum element of the flattened grid after slicing by steps of 3, and we mutate the grid so that we can always use the same slice, avoiding any need for loop variables. Because the grid is always 7×7 , we can use string multiplication with `eval` to do each cell operation 7 times and each row operation 7 times.

List comprehensions

The most common structure of our solutions was to use a list comprehension to transform the grid. The most basic form of the code will look like this:

```
p=lambda g:[F(x)for x in r]for r in g]
```

This operates on single cells only, which is quite limiting. To do more complex operations that require looking beyond a single cell, state variables can be introduced with the walrus `:=` operator. For example, to include the cumulative sum of the row, one can write

```
p=lambda g:[(s:=0)or[F(x,s:=s+x)for x in r]for r in g]
```

Initializing the state variable could sometimes be done more efficiently using `map` and `lambda`:

```
[(a:=0)or[F(x,a:=...)for x in r]for*r,in zip(*g)]
#shortens to
[*map(lambda*r,a=0:[F(x,a:=...)for x in r],*g)]
```

Another technique is mutating the list while operating on it. In particular, `list.pop()` was a source of multiple tricks. For example, it could facilitate “lookahead” in a list comprehension, such as testing whether a certain color appears later in the row:

```
#Original version, check if there is a gray cell in the row to the
#right of the current cell
p=lambda g:[F(x,5in r[i:])for i,x in enumerate(r)]for r in g]
#Optimized with pop()
p=lambda g:[F(r.pop(0),5in r)for _ in r*1]for r in g]
```

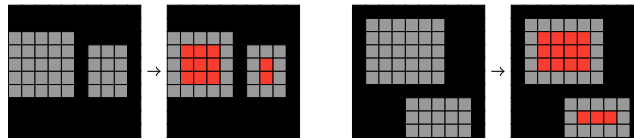
`list.pop()` can also implicitly reverse a list when popping from the back instead of front:

```
p=lambda g:[F(r.pop(0))for _ in r*1][::-1]for r in g]
#shortens to
p=lambda g:[F(r.pop())for _ in r*1]for r in g]
```

Regular expressions

Regular expressions, in particular `re.sub`, are extremely powerful for concisely editing strings. Therefore, it is often shorter to convert the given input matrix into a string, perform the substitution, and convert back to a matrix using `eval`. Our final submission used regex in a total of 46 solutions.

A simple example of regex can be seen for [Task #294](#):



```
import re
p=lambda g:eval(re.sub("(?<=5.{34})5(?={34}5)","2",str(g)))
```

The above substitution replaces all gray pixels with red if they are in the middle of a diagonal of three gray pixels. Since the grid size is fixed (10×10), we know that diagonally connected pixels are 34 characters apart.

For more complex drawing tasks, a *rotate-and-replace* strategy was often used:

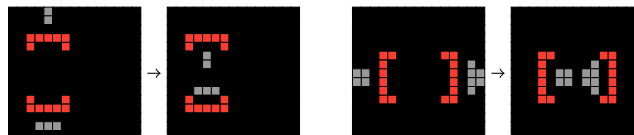
```
p=lambda g:[g:=eval(re.sub("...", "...", f"*zip(*g[::-1]),}"))<+
for _ in g][3]
```

We employed different variations of this approach (list comprehension, recursion, or loops) depending on the task. An interesting trick we found to save one byte is that f-strings allow unpacking, i.e., `f"*zip(*g[::-1]),}"` vs `str([*zip(*g[::-1])])`.

In some of the most complex tasks, we utilized multiple substitutions with different expressions. One way of doing so is:

```
p=lambda g:[g:=eval(re.sub(*s,f"*zip(*g[::-1]),}"))for s in[<+
("...", "...")*16+(("...", "...")*4)][-1]
```

Because the result after `re.sub` is `eval`ed, the replacement could be any Python expression. This was a source for some interesting tricks. For example, in [Task #154](#), we use the replacement `*[\g<0>][::-1]` to reverse the matched substring.



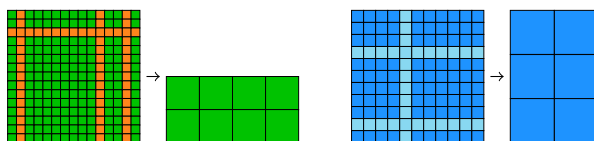
```
import re
p=lambda g,k=0:eval(re.sub("[^(2){9}2"*2+"?", "*[\g<0>][::-1]",<+
f"*zip(*k or p(g,g)),}"))
```

“Dimension” recursion

A conceptually difficult but very powerful technique is to re-use the function `p` for different shapes of input: a 2-d grid (`list[list[int]]`), 1-d row (`list[int]`), or 0-d cell (`int`). This can lead to significant savings in cases where the 2-d operation and 1-d operation can use overlapping code. There are several variations, but one template is

```
p=lambda g:g*0!=0and[p(r)for r in g]or F(g)
```

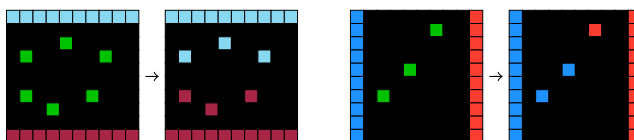
where `F` is the `int -> int` operation. The condition `g*0!=0` checks if the input is a list, and separates the 2-d/1-d cases from the `int` case. For example, [Task #021](#):



```
p=lambda g:g*-1*-1or[p(g:=r)for r in g if g!=r][::2]
```

In this task, the row and column operations can use the exact same code: skip identical consecutive elements and downsample by 2. Note that the formula `g*-1*-1` can be used here instead of `g*0!=0` because cells never have value 0 (black) in this task.

This technique becomes extremely potent when utilizing additional arguments to `p`. For example, [Task #040](#):



```
p=lambda g,h=[]:g*0!=0and[*map(p,g[:1]*5+g[9:]*5,h+g)]or h%~h&g
```

The high-level idea behind this solution is to color each cell according to the nearest corner of the grid. The trick is that recursively applying `g[:1]*5+g[9:]*5` will yield the colors of the nearest corners (taking advantage that the grid is always 10×10). At the end of the recursion, `h` will contain the current cell and `g` will contain the nearest corner. Note that `h%~h&g` is a math trick to efficiently express `h` and `g`.

Guess-and-check

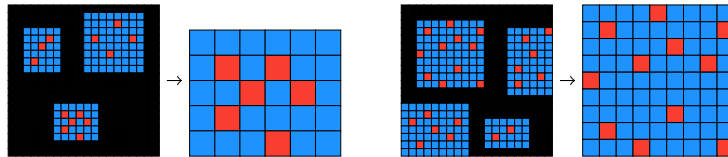
In some problems, it is difficult to search for an object, but easy to verify whether an object is correct. As such, it is more efficient to simply keep guessing the object until the correct one is found. One example is [Task #124](#):



```
p=lambda g,x=0,y=2:(G:=[(i//y*x*[0]+g[i%y])[:10]⊕
for i in range(10))]*(G[:4]<g<G)or p(g,~-x%3,y^1)
```

The goal of this task is to extend a pattern of height 2–3 and a horizontal offset of 0–2. In this solution, we guess the height y and offset x and use $G[:4]<g<G$ to check if it agrees with the input. If the condition is true, the function returns, otherwise it will proceed to the next guess $p(g, \sim x\%3, y^1)$.

Built-ins such as `max` and `min` could also be used to choose the correct guess. For example, **Task #216**:



```
p=lambda g:max((-c:=sum(1:=[r[x%17:x%21]for r in g[x%19:x%22]]⊕
,g).count)(0),c(2),c(1),1)for x in range(8**6))[3]
```

It's difficult to enumerate *only* the blue rectangles; it's much easier to enumerate every rectangular region and check that it's a valid blue rectangle. To enumerate every rectangular region, we use the math trick that $x\%17$, $x\%21$, $x\%19$, $x\%22$ go through every combination of values (by the Chinese Remainder Theorem). To pick out the answer, we use `max` to simultaneously find the most red cells (via `c(2)`) and verify the region is, in fact, a complete blue rectangle, i.e., has no black cells `-c(0)` and cannot be extended with more blue cells `c(1)`.

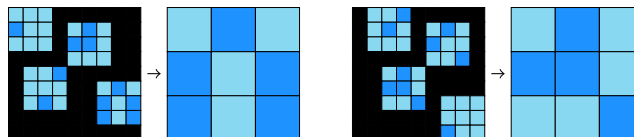
3x3 neighborhood iterator

A compact method to iterate over all 3×3 neighborhoods in the grid is

```
[F(i)for*h,in map(zip,g,g[1:],g[2:]))for*i,in map(zip,h,h[1:],h[2:])]

```

Using `eval/exec`, the repetition in the two `for` clauses can be reduced, as seen in our solution to **Task #271**:



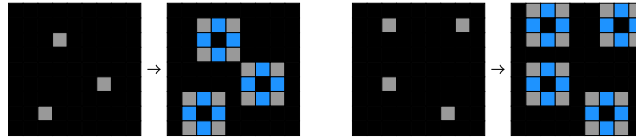
```
exec(f"p=lambda g:max([str(g).count('1'),g['⊕'
for*g,in map(zip,g,g[1:],g[2:]))*2])[1]")
```

pysearch

Many problems required small lookup tables, e.g., mapping of one color to another. We used the utility `pysearch` extensively to find the shortest expressions that reproduced a desired input–output relationship.¹⁷

17. <https://github.com/lynn/pysearch>

A more sophisticated usage of pysearch was to find formulas that could be iterated to produce the desired transformation. One example is [Task #282](#):



Using pysearch with custom code to simulate four rotate-and-transform operations, we found that the desired pattern could be produced with the formula $p//5|p^c$, where p is the previous color (when scanning left-to-right in each row) and c is the current color. With some clever usage of `eval` and slice recursion, our solution is:

```
p=lambda g:g[99:]or[eval(8*"+(x:=r.pop()),x//5|x^0")<b
for*r,in zip(*p(g*2))]
```

Cheese and hardcoding

In code golf parlance, “cheese” refers to solutions that exploit weaknesses in the judge, rather than adhering to the intended solution (similar to the concept of overfitting in machine learning). In this competition, most tasks had hundreds of tests, so opportunities for cheese were limited. However, an example of severe cheese is [Task #346](#):



For human eyes, the intention is obviously to pick the color which has a point surrounded by the other color. But that usually is the least frequent color in the grid, which is much easier to express in Python. By optimizing the slice of the grid to look at, we can force the least frequent color to always match the solution:

```
p=lambda g:[[min(f:=sum(g[:6]+g,[])[11:],key=f.count)]]
```

In addition, we solved two problems by hard coding a lookup table: [Task #048](#) and [Task #319](#). Both of these tasks need only 1 bit to resolve, so hard coding a table of bits was reasonably efficient. (In fact, Task #048 required less than 1 bit per test case, because the answer was more likely to be 8 than 0 by a factor of about 2 : 1, which has an entropy of 0.92 bits.) Our hard-coding technique is quite interesting by itself, using the hash of the grid combined with data stored in a byte string:

```
(o:=hash((b"%a"%g,)))/b'data_goes_here'[o%14]%2
```

Extra bytes can be added to the hashed data `b"%a"%g` to optimize the efficiency of the lookup table, which can be searched by brute force.

zlib compression

Using a combination of Python’s `zlib` and `exec`, it is possible to run compressed code. This technique was useful for solutions longer than about 200 bytes. To get the best possible compression, we had two pieces of infrastructure:

1. We used `zopfli` or `libdeflate` for an initial pass of compression. However, these tools do not account for the extra cost of embedding data into a Python script, i.e., certain bytes must be escaped. We wrote a re-encoder that used the same Huffman tree from `zopfli` or `libdeflate`, and then used dynamic programming to find the optimal encoding.
2. We used ad-hoc scripts to try different combinations of variable names to find what compressed best. Golfing with compression completely changed the techniques for optimal code size. Compression would make code repetition almost free, so the goal with `zlib` golfing is to use the same constructs over and over, rather than trying to find the shortest construct for each step of the algorithm.

No shell

It is worth mentioning that although shell commands were allowed in the competition (e.g. `os.popen`), and likely would have been useful in certain tasks (e.g., `perl` was available and is an excellent golfing language), we did not use this technique as per mutual agreement among the teams in the top 3. The reason being that the legality of this technique was clarified somewhat late in the competition, and fully exploring this technique would add considerably more work to an already large workload.

B.1.3. ACKNOWLEDGEMENTS

We are grateful to [@garrymoss’s *Golfed solutions and explanations to hard problems*](#),¹⁸ [@kenkrige’s *Chipping skills 3: Regex*](#),¹⁹ and the commenters there for sharing solutions that improved on our best at the time. We would also like to thank everyone who contributed to the public scores spreadsheet, which was an invaluable source of score benchmarks.

B.2. jailctf merger (2nd Place)

B.2.1. SUMMARY

Our team was formed from 5 CTFers involved in the organization of jailCTF,²⁰ one of whom is a seasoned code golfer. We focused on getting tooling for compression and easy collaboration very early on the competition, hoping that a team effort could make up for the lack of golfing experience the other four of us had. In the last month, we wrote a fuzzer to get some easy bytes on some tasks, which ended up finding a cool trick and helped save 100+ bytes in the final stretch.

The work was approximately split as such:

- **lydxn** did most of the heavy code golfing, finding the majority of the tricks in our solutions

18. <https://kaggle.com/code/garrymoss/golfed-solutions-and-explanations-to-hard-problems>

19. <https://kaggle.com/code/kenkrige/chipping-skills-3-regex>

20. <https://ctf.pyjail.club/>

- **quasar098** worked on much of the tooling
- **oh_word** focused on compression and compression based tasks
- **hellopir2** did a lot of optimization all around
- **Quasar-0147** was mostly inactive

Submission and Tooling

All of the code and tooling that we used can be found in our online repositories,²¹ including **charbrute** and **var_genetic**.²² Our final **submission.zip** file is also available,²³ along with the **spreadsheet** we used as a central hub for information.

B.2.2. CHARBRUTE

We named the fuzzer **charbrute**, because what it does is randomly mutate the code at the character level (deletions, substitutions, insertions). A so-called “astbrute” was proposed but never implemented due to high complexity and low perceived gains. The fuzzer was originally pretty bad, and only handled deletions and substitutions. Over time, we added a few improvements, the major ones being:

- Handling insertions as a replacement for substitutions
- Adding a chunk mover which moved around chunks of code (this one didn’t really give good results, maybe moving it to before the other mutations would’ve helped)
- Forcing the spot mutations to be centralized around a hotspot with a probability distribution
- Using pypy, since pypy3.11 came out recently. This creates a few minor differences such as set ordering and float caching but these can be manually filtered or blacklisted.

It mostly found minor saves and optimizations in our code, but the occasional large save was found. It very quickly got saturated for our code dataset, and so we were forced to keep adding new features or slight modifications to keep its productivity. Examples of some saves that it found are below:

```
# task 131, -1b
p=lambda g:exec("c=2;g[:]=zip(*([j for*j,in g if(c:=c-(s:=max(j))<=
*(c>0))|s|+[[8]*9]+[[0]*9]*99)[len(g)-1::-1]);"*4)or g
p=lambda g:exec("c=2;g[:]=zip(*([j for*j,in g if(c:=c-(s:=max(j))<=
*(c>0))|s|+[[8]*9]+[g[0]]*99)[len(g)-1::-1]);"*4)or g
# human optimization, -1b
p=lambda g:exec("c=2;g[:]=zip(*([j for*j,in g if(c:=c-(s:=max(j))<=
*(c>0))|s|+[[8]*9]+g[:1]*99)[len(g)-1::-1]);"*4)or g

# task 125, -3b
```

21. <https://github.com/orgs/jailctf-merger-goog-golf/repositories>

22. <https://github.com/jailctf-merger-goog-golf/compression>

23. <https://github.com/jailctf-merger-goog-golf/golf/blob/master/submission.zip>

```

p=lambda g,i=87,q=8:g*-i or [[12%c,~q%7//c*8,-(q&(q:=c)%3)%5]↵
[i//42]or c for c in g]for g[::-1]in zip(*p(g,i-1))]
p=lambda g,i=87,q=8:g*-i or [[12%c,q//c*c,-(q&(q:=c)%3)%5]↵
[i//42]or c for c in g]for g[::-1]in zip(*p(g,i-1))]

# task 196, -3b
p=lambda g,i=23:g*-i or [[q:=[c|4-c&6%~q,c|-c&~q%3,c%4] [i//8]↵
for c in g]for g[::-1]in zip(*p(g,i-1))if(q:=8)]
p=lambda g,i=23:g*-i or [[q:=[c|8-c&q,c|-c&~q%3,c%4] [i//8]↵
for c in g]for g[::-1]in zip(*p(g,i-1))if(q:=8)]

```

The most notable trick we fuzzed is what I call “pop rotation”. For some context, shortly after we matched the public best on [Task #015](#), which was 93 bytes at the time, it looked like this:

```

p=lambda g,i=-7:g*i or p([[r.pop()%9|36%(6^-[0,*r] [i//7]))%13↵
for _ in g]for*r,in zip(*g)],i+1)

```

Note the unique inside-out recursion structure with the `p` call on the outside (which was rarely done because putting the recursion on the inside saved 1 whitespace character), and the squareness of all the input grids.

The fuzzer found this:

```

p=lambda g,i=-7:g*i or p([[r.pop()%9|36%(6^-[0,*r] [i//7]))%13↵
for r in g]for*r,in zip(g)],i+1)

```

And it turns out you can just completely remove the `zip()` for the current known best of **87 bytes**:

```

p=lambda g,i=-7:g*i or p([[r.pop()%9|36%(6^-[0,*r] [i//7]))%13↵
for r in g]for*r,in g],i+1)

```

This is pop rotation, where you perform a 90 degree counterclockwise rotation of a grid with just `.pop()`! For comparison, let’s see the other “rotate a grid 90 degrees in a list comprehension” formats:

```

# square grid:
p=lambda g:[r.pop()for r in g]for r in g]
p=lambda g:[r.pop()for _ in g]for*r,in zip(*g)]
p=lambda g:[[c for c in g]for g[::-1]in zip(*g)]

# rectangular grid:
p=lambda g:[r.pop()for r in g]for r in g[0]*1]
p=lambda g:[r.pop()for _ in r*1]for*r,in zip(*g)]
p=lambda g:[[c for c in r]for r in zip(*g)][::-1]

```

Of course, it has its own advantages and disadvantages compared to the other typical rotation formats, which is why it is not able to be used everywhere.

In the end, we applied this on these tasks:

Task #002	86 bytes
Task #015	87 bytes
Task #086	154 bytes
Task #095	70 bytes
Task #220	82 bytes
Task #230	95 bytes
Task #251	84 bytes
Task #272	87 bytes
Task #283	79 bytes
Task #336	86 bytes
Task #352	82 bytes

B.2.3. COMPRESSION

For compression, we utilized a variable rename optimization script, similar to the public ones. There are two key differences though:

1. Stripping the header and checksum and using `wbits=-9` in the decompression saves a few bytes.
2. Using a genetic algorithm (implemented by Gemini) to optimize for raw compressed length, compared to naive brute force.

Overall, these optimizations saved a few bytes on each compressed task that we had, which was down to just 24 by the end of the competition.

Our initial approach to the longer tasks was to get an LLM to generate an initial working solution by prompting it with the generation code, general code golfing tips, and some human ideas for the specific task at hand, and then afterwards optimizing the code manually for both compressibility and length. However, most of the llm solutions we golfed got obsoleted by either public notebooks or with a solution rewrite.

Other than that, we had a few main strategies, either to rewrite the code for compressibility/brevity or to repeat/reuse code to do different subtasks. We abused the second strategy quite a lot more than the first. In fact, out of every compressed diamond, we own the two highest compression ratios of ~2.91 on our **Task #044** diamond and ~2.46 on our **Task #173** diamond. Some examples of compression optimizations we found are below:

```
# "for r in " substring repeating during rotation
[r for*r,in zip(*g)][::-1]
[[*r]for r in zip(*g)][::-1]

# "g[y]" substring repeating during len checks
len(g[0])
len(g[y])

# "_ in range(len(_))" substring repeating during bounds checks
len(g)>y>-1<x<len(g[y])
y in range(len(g))!=x in range(len(g[y]))

# optimizing the huffman tree
```

```

x>y
x<y

# renaming variables to make your code more readable
[[c for c in r]for r in g]
[[g for g in g]for g in g]

# adding whitespace in the correct places
}for i in range(r)
} for i in range(r)

```

Note that these tips are not always positive changes! Mileage varies depending on the contents of the rest of the code. As such, optimizing for compressed length is a lot of trial and error (and pattern recognition), and when each compression attempt takes a couple minutes to brute force an optimized variable renamed version to check if it is shorter, it becomes very annoying.

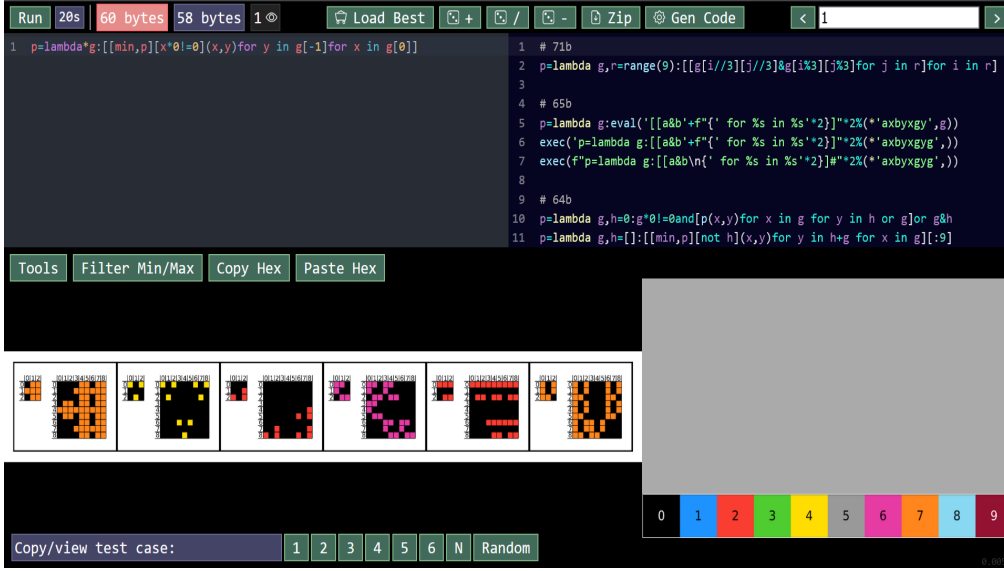
However, the one benefit of compression golfing is that since no one started out particularly experienced at this, our team wasn't at a disadvantage with respect to this golfing skill. But it is quite annoying and very easy to waste a lot of time on if you don't approach the task correctly. Our team had many attempted rewrites that turned out to be a lot longer when compressed compared to the existing solutions, even if they were potentially shorter in terms of uncompressed length.

B.2.4. WEB UI

We created a centralized web server(s) for golfing and storing solutions, which the other top two teams did not do. This involved a flask server and a websocket server. We did not have a git repository storing all of our solutions because that is too annoying to handle.

Features of the web UI + websocket server that promoted collaboration and efficiency:

- Store solutions and annotations (i.e. per-task notes)
- Real time updates to both solutions and annotations (using websockets)
- Basic authentication to prevent intruders
- Copy test cases button for easy regex manufacturing in regex101 or other
- View test cases button to see how the manually written test cases are wrong compared to the generated ones
- Gen code button to view test case generation code per task
- 20s timeout changeable to 90s timeout to prevent server strain by default
- Two-way integration with our central spreadsheet (google apps script) for easier analysis.
- Random buttons for golfable task finding
- Other tools integration

Figure 6: A screenshot of the web UI employed by *jailctf merger*.

B.2.5. PYSEARCH

Like several other teams, we used the `pysearch` tool to brute force arithmetic expressions. A good example of this is [Task #006](#), where we found an optimal 13-byte expression which isn't possible to find by hand:

```
# 43b (gold)
p=lambda g:[[109//c&12%c+7for c in g[0]]]*3
```

`pysearch` tends to work well up to length 13/14 until it starts to become too slow. We tried to utilize `pysearch` as much as possible on any expressions below or around that length. In some cases when the expression is too long to brute force, we can also perform a “partial” `pysearch` on parts of the expression.

After running `pysearch` on several tasks, we realized that specifying `INPUTS` and `GOAL` fields manually became extremely annoying and started generating the parameters programmatically. Our gen script for [Task #293](#) looked like this:

```
from utils import pysearch
import json
import re

task_num = 293

with open(f'../data/training/task{task_num:03d}.json', 'r') as f:
    data = json.load(f)

testcases = data['train'] + data['test'] + data['arc-gen']

search = set()
```

```

for _, data in enumerate(testcases):
    g = data['input']

    for r in g:
        for c in g[0]:
            b = +(c in r)
            x = r[0]
            ans = c-c*b or x or c
            search.add((c, b, x, ans))

pysearch(search)

```

Based on the testcases, we fill a set called `search` containing tuples in the format `(input_1, input_2, ..., input_n, goal)` and pass that into a “utils” script that outputs Rust code to plug into `params.rs`:

```

def pysearch(search, min_uses=1, max_uses=255, varnames=()):
    values = list(map(list, zip(*search)))

    big_template = 'pub const INPUTS: &[Input] = &[%s\n];'

    template = '''
Input {
    name: "%s",
    vec: &%r,
    min_uses: %d,
    max_uses: %d,
},'''

    output = ''
    for i, value in enumerate(values[:-1]):
        if not varnames:
            c = chr(i + 97)
        else:
            c = varnames.pop(0)
        output += template % (c, value, min_uses, max_uses)
    print(big_template % output)
    print('pub const GOAL: &[Num] = &%r;' % values[-1])

```

`c-c*b or x or c` was the formula we found by hand, while the shortest one is actually `b**c*x or c`. We basically reused this template at every opportunity for easy pysearch gains (which was a lot of tasks!). This was especially helpful for the “grid rotation” tasks, e.g. [Task #002](#) / [Task #243](#).

B.2.6. OTHER TOOLING

One useful tool we found online was called `infgen` and it allowed us to debug raw deflate streams, seeing both how many bytes each zlib mini-segment used and also how the substitutions were taking place in ways that we might not be able to intuitively see.²⁴ Another

24. <https://github.com/madler/infgen>

utility we made was `autogolf`, which made sure we didn't do any silly things with excess spacing or parenthesis and such. This tool was a heavy modification of `ast.unparse`, but with golfing in mind. We integrated both of these to the golf server as buttons to allow us to easily use them.

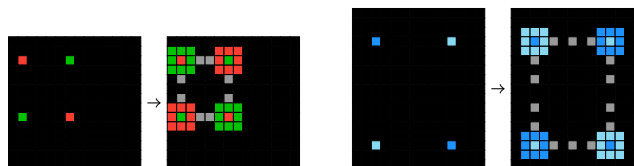
B.2.7. INTERESTING TASKS

Task #048

This was the only task where we hardcoded every test case using `hash()`, as described elsewhere in a detailed blog post.²⁵ This trick could also be applied to Task #319 to find the right color.

Task #387

This task has you surround the nonzero colors with the other nonzero color, as well as connecting the pixels with an alternating strip of gray and black pixels.



Looking at the other teams' solutions, we were the only team to use regex to solve the task, with it beating out the non-regex solutions of all teams but Code Golf International. So, we thought it would be interesting to explain the regex approach we came up with.

Firstly, notice that this task, like 99% of the tasks, can be greatly simplified if you only deal with one orientation at a time and then rotate. But for this task, the question of how you handle each orientation is not so easily answered. The first thought might be to do something as follows, where you complete one 3×3 and one edge at a time:

```

---      ---      BBB      ---
_A-----B_ --> BAB5_5_5_B_
---      ---      BBB      ---

```

However, what's interesting is that this proposal actually fills 2 3×3 s and 2 edges on the initial grid! This gives a new idea: since each 3×3 and edge is going to be matched twice in the full rotation, we should try to fill in only half of an edge and half of a corner on each passthrough.

The edges are more apparent in how to approach than the 3×3 s. Rather than filling the whole space between the two dots, you only need to fill it halfway. This turns our first transformation into something similar to the below:

```

---      ---      BBB      ---
_A-----B_ --> BAB5_5_5_B_
---      ---      BBB      ---

```

25. <https://lydxn.github.io/posts/fun-with-magic-hashes-in-python/>

The 3×3 s are a little trickier in the approach. If we stare at it a little, we can see that we have a few options in which pair of opposite corners and which pair of opposite edges we want to fill in on the first pass.

proposal 1	proposal 2
<pre> --- --- B__ --- _A_____B_ --> BAB5_5__B_ --- --- __B --- </pre>	<pre> --- --- BB_ --- _A_____B_ --> _A_5_5__B_ --- --- _BB --- </pre>
proposal 3	proposal 4
<pre> --- --- __B --- _A_____B_ --> BAB5_5__B_ --- --- B__ --- </pre>	<pre> --- --- _BB --- _A_____B_ --> _A_5_5__B_ --- --- BB_ --- </pre>

Hold on, aren't we doing this in regex? Before we decide which proposal is the best, we need to figure out how to implement this in regex.

So the basic idea we had to implement this was like this:

```
[(stuff)___    ___
[(stuff) _A_____B_
[(stuff)___    ___
```

Notice how since the rectangular frame is the only thing on the grid, all of the `(stuff)` groups should be equal. This fixes the issue that we don't know the distance from `A` to `B` which would otherwise make drawing the half dotted line from `A` to `B` quite annoying when trying to access the row below. Instead, if we capture `(stuff)`, we can just go to the next line by doing something like `[^\n]+` and then matching `[(stuff)` to find the cells below `A`.

From here, there's not that large of a difference between the methods, so I picked the first one because it was the easiest to implement and probably slightly shorter. After a bit of golfing, we arrive at this 201 byte regex solution:

```
import re
p=lambda g:[g:=eval(re.sub(r'([~]*)0(?:[~]*1)0(, ([^0]), )0,([~+
~]*)?(?!0|1|4)((\d).*?1.{6})0',r'\1\7\2\7\3\7',[c:=1|1+
en([5])//3])+[5][c:],\6\7',f'{*zip(*g),}'))[:-1]for _ in g][3]
```

Let's break down how this works:

- Firstly, the `([^\n]*)` captures all characters between a `(` and a `\n` on a single row. This means `\1` is the `(stuff)` that we wanted to match from earlier. The `\n` is consumed because it will be replaced with `\n` in the replacement.
- Then, the `([^\n]*\n)` captures all characters until the `\n` below the previously consumed `\n`. This `\n` is again consumed because it will be replaced with the `\n`. Following this `\n`, we expect `, A,` in the row, which is matched with `(, ([^\n]*), )`, making `A` be group `\4`. The following `\n`, is consumed, again to be replaced with `\n`, later.

- After this 0, we expect the B cell to be somewhere in the row. All the cells between the 0 and the B are captured with `([^\]]*)`. The regex ensures the color B exists by searching for the last non-zero, non-A color by using `(?!0|\4)(\d)`. Note that this never captures a 5 instead of B because `*` is greedy.
- Then it simply captures up to the next (stuff) group using a lazy `.*?\1`, and then consumes the third 0 to be replaced with B, matching with the B on the third row of proposal 1.
- The replacement is mostly just putting back everything that was consumed and replacing 0s with Bs, but there is a small pysearch slice for replacing the first half of the cells between the 0 and B on the middle row with the alternating 5, 0 pattern, which will get eval'd into the correct replacement.

There are slight variations to this regex that are the same length, but we never found anything shorter than 201 bytes.

Task #023

In this task, light blue 2×2 squares and red 1×3 lines were placed to fill gray slots on the grid.



This task was tricky because there are many possible positions for the squares and lines but only one would work. We found that the easiest way to solve this task was to use a recursive depth-first search. Essentially, each call of the recursive function would find a suitable gray square, and check all shapes (1×3 , 2×2 , 3×1) at that location to see if they fit. Then the algorithm would call itself with the shape retained, and eventually this would fill up the whole grid.

This is one example of a common theme in golfed solutions: **the most efficient example is rarely the shortest**. There were no backtracking checks for 1×1 gray squares which would obviously mean no red or light blue shape could fit there, since that check would take up extra bytes. We also had to optimize for compression, since our solution was so long. To get the gray square, we found that simply getting the topmost leftmost gray square using two loops would work since all previous shapes would be filled in going top-down left-right as well, so the optimal solution would always be found.

The shapes are encoded as bytestrings, and unpacked using `*x,y` where `y` is the color of the shape and `x` is the list of positive offsets, encoded using modulo. We used byte strings and modulo to get the position offsets to be as short as possible. Also, `try except` was a short way to cancel the recursion if the shape did not fit into the gray slots, so we used that, along with a non-existent variable, to cancel the loop. Lastly, bitwise operators allowed us to reuse the number 4 a few times which would align nicely with the huffman tree during compression.

```
def p(i):
```

```

for u,m in enumerate(i):
    for r,f in enumerate(m):
        for *e,m in b"lur",b"lpt",b"lpum":
            try:
                for t in e:i[u+t%4][r+t%3]&4or n
                for t in e:i[u+t%4][r+t%3]=m
                if p(i):return i
                for t in e:i[u+t%4][r+t%3]=5
            except:i
            if f&4:return
        return i

```

We originally had the above which got compressed with our genetic varname compressor to be 198b. However, one more byte could be saved by using bytestring bytes that aligned more nicely with the existing code (in huffman tree). The exact bytes and offsets (see %4 and %5 below) were brute forced for ones that seemed good.

```

def p(e):
    for n,t in enumerate(e):
        for s,r in enumerate(t):
            for *b,t in b"(in",b"(tp",b"(ite":
                try:
                    for i in b:e[n+i%4][s+i%5]&4or y
                    for i in b:e[n+i%4][s+i%5]=t
                    if p(e):return e
                    for i in b:e[n+i%4][s+i%5]=5
                except:e
                if r&4:return
            return e

```

Other teams were able to get their solution shorter using regex, which we did not end up using for this task.

B.3. ox jam! (3rd Place)

Our team managed to finish 3rd place in this competition. The five of us are a pretty international team, coming from four different continents.

B.3.1. COMPETITION OVERVIEW

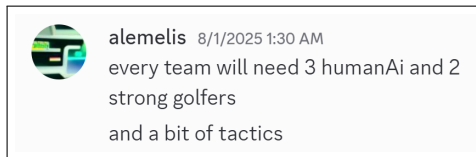
As has been discussed extensively in prior sections, our goal was to create Python functions to solve all four-hundred problems in the ARC-AGI-1 training dataset, and (of course) to do so using the shortest possible solutions, minimizing total byte size with no sharing of code between tasks. Our overall approach — which employed no Artificial Intelligence whatsoever — can be summarized as follows:

- Manual solving and optimization by human experts, using various code golf tricks that are described in depth on the Kaggle discussion forum and in many of the write-ups produced by different teams.

- A collaboration pipeline we set up through automated Git workflows, allowing us to quickly collaborate and make improvements under the pretty significant time constraints that we faced.
- Similar to other teams, some automated search for improved size over zlib compression.

B.3.2. TEAM EXPERTISE

Collectively, the five of us have just over three decades of experience code golfing.²⁶ Although the hobby has been around for a long time, this was probably the single biggest competition there’s ever been, and so we’ve spent many years solving similar problems for mere recreation without looking for the big sort of pay in tournaments like this one. It’s our impression that between the top teams, there was really not that much variance in the tricks that we used; nearly all of us know how to do the rotations, the pops, splits, zlib compressions, etc. However, perhaps one major distinction (in addition to raw effort and hard work) was the sense of a “golfer’s instinct.” We’ll touch upon this topic later when discussing code golf as a global optimization problem, but it basically boils down to how one decides (when there are multiple possible paths to optimize) which one is the *right* one. Regarding team structure, we highlight below one particularly relevant message posted within hours of the competition’s launch, speculating that any team wanting to seriously compete would need three (as they referred to it) “human AI members” — basically, people whose goal was to just *grind, grind, grind*, because 400 code golf problems within three months is no light work.



While many other teams were able to supplement that with Artificial intelligence, *none* of us in the top three used AI to produce any of our solutions.

B.3.3. SCORE TIMELINE

In Figure 7(a) we display a timeline of our team’s score throughout all three months of the contest. One can clearly observe a huge jump toward the beginning, when we went from zero problems solved up to four-hundred. Within the first two weeks of this twelve-week competition, we had solutions posted to all 400 problems. From then on (over the next two-and-a-half months), our time was spent hunting down any small optimizations we could find. At least on this scale, it may appear that basically no real improvement was made during the last two months of the competition — and yet, this is where all the serious back-and-forth occurred between the top teams. To zoom in further on that section, Figure 7(b) shows our score on a smaller range after we had produced our initial four-hundred solutions. Once again, almost all of our improvements occurred very quickly, with seemingly little progress

26. Links to our [code.golf](#) user profiles: {[ovs-code](#), [MeWhenI](#), [GrayJoKing](#), [xsot](#), [zacjl](#)}.

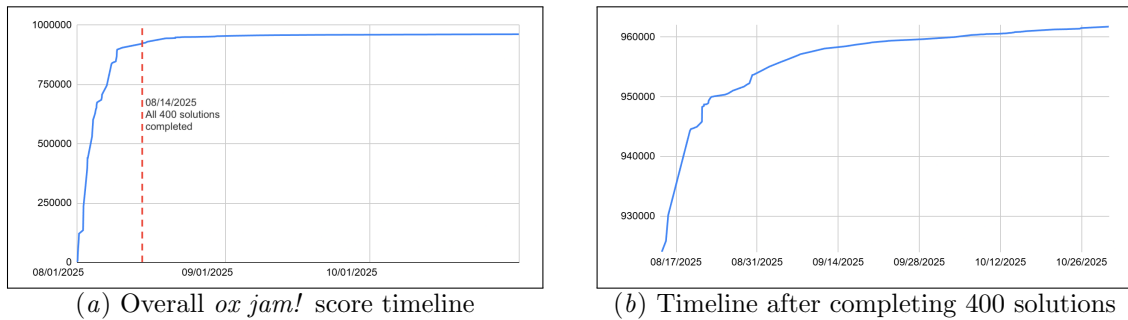


Figure 7: Score timelines for *ox jam!* spanning different segments of the competition.

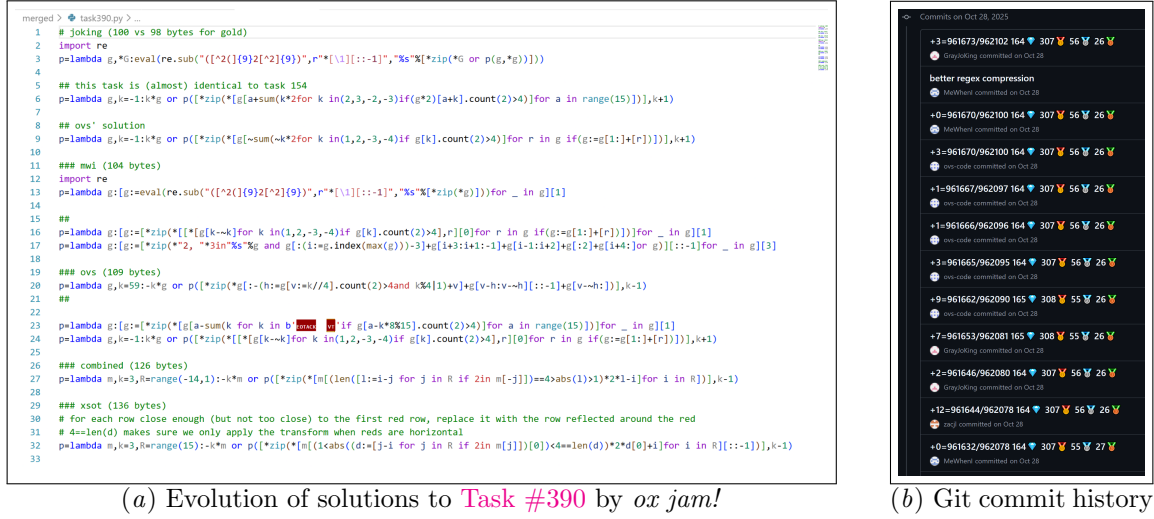
made over the last few weeks. It was the activity during these final days that ultimately determined who came in first, second, and third.

B.3.4. WORKFLOW PIPELINE FOR COLLABORATIVE GLOBAL OPTIMIZATION

Team code golfing isn’t something that happens too often. One major challenge we encountered early on was not just in applying several small golfs, but also how to consider entirely different methodologies when approaching a given problem. In response, we developed a series of GitHub workflows that allowed us all to work individually, and then compile our solutions into a single document for each problem where we could explore our own improvements along with the ideas of everyone else simultaneously. In Figure 8(a), we illustrate one such document for **Task #390**, where every line shows a different solution and/or different approach. Some of these are small improvements on each other, but others reveal completely different approaches. Hence, one of the interesting questions that arises when treating this as an optimization problem is considering not just “*how can I pick the top solution and improve it?*” but also “*is this even the solution that I should be dedicating my time to improving?*” As for how best to answer that question, we believe it invariably comes down to instinct and expertise. This is something our teammates have gotten rather good at figuring out over time: even though one solution may currently be the shortest, there may actually be more opportunity for improvement in another solution altogether. What that means for AI — and whether it can eventually match human expertise — is something we’ll cover in subsequent sections, but we believe it significantly helped set us apart. Figure 8(b) shows a screenshot of our GitHub commits over time, all of which were pushed using an automated system that would track our golf medals (e.g., diamonds, golds, etc.) along with our total score and what we thought the best possible score might be.

B.3.5. COMMUNITY-DRIVEN OPTIMIZATION

Another important aspect of this competition was the emphasis on community-driven optimization, and especially the unofficial score spreadsheet (see Figure 9) that emerged early on. Coming from the code golf community, we recognized that an incredibly valuable piece of information is knowing how well other people are performing. Very often you’ll see a problem where it seems like an optimal score has been reached (e.g., 70 bytes), but then

Figure 8: Artifacts of the workflow pipeline employed by *ox jam!*

one person breaks through and finds a 69-byte solution, and within an hour somebody else has submitted a 67 or a 66. Just knowing the *potential* to improve is often what it takes to push golfers in the right direction, i.e. to let them know where they should dedicate their time. And so, we are really thankful to the broader code golf community for coming together and forming this resource. However, a lot of us on the top ultimately realized that (unfortunately) as much as we wanted to continue sharing this information, if this information wasn't *obligated* to be shared by everyone, we each would be putting ourselves at a competitive disadvantage. So, toward the end, there were fewer updates to this doc, but it was still great for giving us all a sense of which puzzles to approach and where potential optimizations might lie.

We did feel there were a couple pain points in this competition that made it somewhat difficult to participate and commit ourselves fully. Many other top golfers probably could have come in and contributed even better solutions than we'd seen, but instead elected to opt-out due to various troubles (e.g., the lack of a single-hole leaderboard). If a similar contest happens again in the future, the code golf community can likely offer great ideas about how to run such events — perhaps resulting in even wider participation beyond the thousands of teams that signed up this year.

B.3.6. LOOKING FORWARD: TRACTABILITY OF CODE GOLF FOR AI

As noted earlier, all top three teams used no AI in their solutions, making this a bit of an oddball competition in the context of NeurIPS. In Figure 10, we chart the total program lengths of the top ten teams; while teams 10th through 4th all scored in roughly the same range, there is a noticeable drop in byte count between this group and the top three. A couple factors may explain this gap: one reason (we theorize) is the prize pool being equivalent for 4th through 10th place, resulting in less competitive pressure for teams who felt they couldn't break into the top three. It also suggests there may have been only about

THE 2025 GOOGLE CODE GOLF CHAMPIONSHIP

Code Golf Championship Scores

File Edit View Insert Format Data Tools Extensions Help

Menu

100%

\$ % & #

Arial

12

Figure 9: The (unofficial) community score spreadsheet initiated by 4atj.

three teams worth of people who had both the expertise and the time to really commit and see this contest through.

As for how Artificial Intelligence might eventually be able to match this performance (given that such a huge gap exists between human experts and other teams), it’s worth noting that at conclusion of this competition, we now have probably one of the highest-quality code golf datasets that has ever existed. However, for anyone aiming to use these datasets in training AI, we would stress that (beyond the final solutions themselves) they should also consider the Git histories and their ability to track the improvement to solutions over time. This can help to shed light on which approaches turned out to be dead ends, aiding in the development of the aforementioned “golfer’s sense” of not just good tricks, but also what approaches to avoid.

At the time of this writing, the ARC Prize 2025 also recently concluded, which is likely of interest to those who are curious about this golf competition. Among the resulting write-ups, the first place solution was extremely illuminating,²⁷ discussing issues such as (a) the creation of synthetic data to expand the available set of training inputs, (b) additional ideas on how such problems might be approached, and (c) the value of not just code solutions, but also natural language (i.e., English language) descriptions, and the huge amount of value that was able to provide. So, while AI largely struggles to perform well in golfing, it nevertheless exhibits a surprising ability to take these very ugly looking code snippets — which, to someone without a lot of experience in the field, might just look like gobbledygook — and provide an excellent explanation of what they mean, along with an exposition of how they work.

27. <https://kaggle.com/competitions/arc-prize-2025/writeups/nvarc>

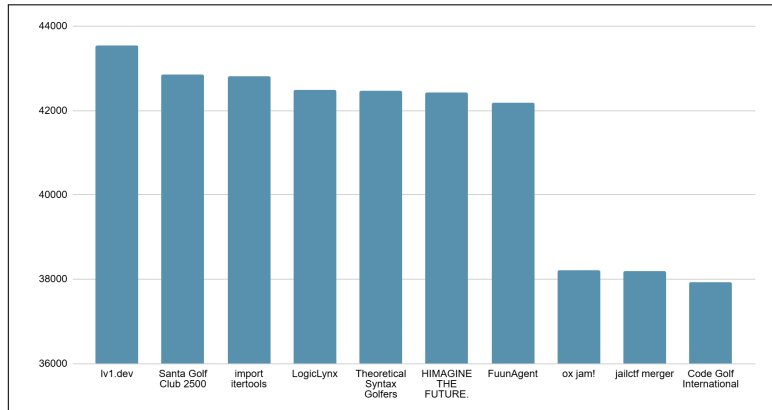
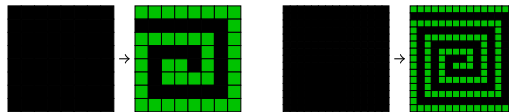


Figure 10: Total program lengths of the top ten teams (in bytes).

B.3.7. APPROACHING EXPERT INTUITION: CONTEXTUAL RECALL

Finally, coming back to the idea of expert intuition and a “golfer’s sense,” there is incredible importance to the idea of *contextual recall*. One such example that demonstrates how our team members were able to achieve top scores for these problems is [Task #58](#), a relatively straightforward puzzle that (given a blank grid) requires the generation of a green spiral:



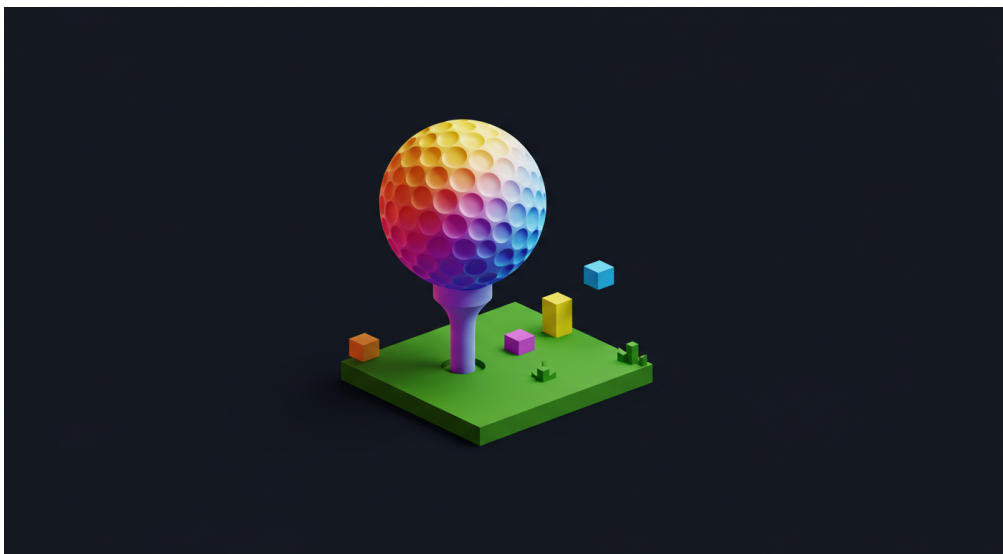
Even if someone knew all the golf tricks and could conceivably tackle this problem from scratch, what ultimately allowed one of our team members to find the shortest program length for this puzzle was his memory of solving a somewhat similar problem in a completely different context where he had golfed the following number spiral:²⁸

0	1	2	3	4	5	6	7	8	9
35	36	37	38	39	40	41	42	43	10
34	63	64	65	66	67	68	69	44	11
33	62	83	84	85	86	87	70	45	12
32	61	82	95	96	97	88	71	46	13
31	60	81	94	99	98	89	72	47	14
30	59	80	93	92	91	90	73	48	15
29	58	79	78	77	76	75	74	49	16
28	57	56	55	54	53	52	51	50	17
27	26	25	24	23	22	21	20	19	18

So, the ability to rely on contextual recall — i.e., to not just have a huge database of tricks, but to be able to answer the questions of *what is this similar to?*, *what is the general structure of this?*, and *what other problems does it look like?* — is probably huge in allowing both Artificial Intelligence *and* humans to escape local minima.

28. <https://code.golf/number-spiral>

Appendix C. Official Competition Logo



Appendix D. Task-Specific Statistics & Shortest Programs

In the pages that follow, we report final byte counts across all winning teams for each of the 400 puzzles in the ARC-AGI-1 training set, along with the shortest program submitted for every task. All solutions shown were produced by one of the top ten teams, with the following exceptions:

- cam jam! (11th place), who contributed a diamond solution to [Task #278](#).
- Dark AGI (13th place), who contributed diamond solutions to [Task #332](#), [Task #349](#), and [Task #354](#).
- Golfers Anonymous (17th place), who contributed a diamond solution to [Task #277](#).
- Ravi Annaswamy (25th place), who contributed a diamond solution to [Task #154](#).

Cell background colors represent golf medals earned — [diamond](#), [gold](#), [silver](#), [bronze](#) — or program format ([uncompressed](#) vs. [compressed](#)).

[illegible]

[illegible]

TASK #	TASK ID	TEAM #1	TEAM #2	TEAM #3	TEAM #4	TEAM #5	TEAM #6	TEAM #7	TEAM #8	TEAM #9	TEAM #10	
#055	272195f4	79	79	77	83	89	94	82	85	93	91	p=lambda i,z=0:i%0!=0 and (p.y,3*(z:=x*(y>1))) for y in i1 or i or 2222096>>2&7
#056	27a28665	40	40	40	44	40	41	39	42	40	40	p=lambda g:[2*(g[2]<11)*3+(g<111)]
#057	286f18e6	48	48	49	51	54	51	54	51	51	51	p=lambda g:[f+1 for any zip(*g[8:]) or p(g*2)*2)]
#058	28673e20	86	88	85	89	90	103	99	103	99	102	p=lambda a:a and len(a)*[3],[a[0],3>>2/len(a),3][2:],*zip(*p[(fzip(*a[2:])))[:0]]
#059	29623171	151	155	153	163	169	174	163	178	166	173	R=range(11);p=lambda g,n=36,o=0:[(g[1][j]==6)56or sum(S:=lg[1&12][k/3][&12][k/3]for k in R[:9])>n=[o:=1]*max(b,(S)for j in R)for i in R]o or p(g,n-1)
#060	29c11459	47	47	47	48	48	48	47	47	47	48	(S)for j in R]for i in R]o or p(g,n-1)
#061	29c67d0e	63	62	63	62	63	63	62	63	63	63	p=lambda g,q=range(8):[(i+j)%max(g)[1]+i for j in g]
#062	2bce6788	123	127	121	162	145	146	185	154	144	140	p=lambda g:exec('q=[]\nfor x in zip(*g):q+=q[:0]*(x*(x-2))=(3)<(q[-1])or[(v or 3 for v in r])\nq[::]=q[9::-0] b,*8) or g
#063	2bce17d4	74	76	75	77	79	78	77	78	79	79	p=lambda g:[k[3>>x+sum(r[1:-any(c)])for _,*c,_x in zip(*g,r)]for x in g]
#064	2c608a1f	135	146	152	174	164	181	169	155	170	154	p=lambda g,n=7:n*g or p([(p:=x:r.pop(),p].count(max((x+r*p)*g[0].count(x)>3).n)-(x,p))>2]for _ in g]for x,r,i,b in zip(*g)if p:=0]]n+2)
#065	2d6579a4	82	84	84	102	107	105	102	109	103	106	p=lambda g:map(g,min(g,key=g.count)if g[3:]else[map(p,*g,*b[1: len(b)//2+1:]for b in g])
#066	2dd70a9a	240	237	244	266	282	257	250	250	301	327	def f(c,n,e,l,d): if len(c)>n+1>v[r[e][n%3]:return f(c,n+4,e,l,d) if r[e][n%3]:return f([(fn]for n in zip(*v)),e,n-d,l-1,1)or f([(fn]for n in zip(*v)),e,n-d,l-1,-1)if l else(3>r<b[e][n])*
#067	2dee19d4	33	33	33	33	33	33	33	33	33	33	def p(c):(n,e),(l,d)=(n,e)for n in range(len(c))for e in range(len(c))if r[e][n%2]:return f([(n]for n in r],n<b,e,2,1)or f(c,n,e,2,-1)if 1>n else[fn]for n in zip(*p([(n]for n in zip(*v))]))
#068	31ae019c	99	112	103	122	129	117	116	128	119	122	p=lambda g:[r[1: len(g)]for r in g]
#069	321b1f66	148	143	150	158	168	147	155	181	161	175	def p(g): b=aasum(g,[]):i=j=100 while i>1:1,a[i]==g=exec("if b[j]:j=j-1%8:g+=j;a[i-g[10]+j]=b[j];a[j]=0\n") return*zip(*lver(a))*10,
#070	32597951	80	78	82	100	92	89	112	87	100	89	p=lambda g:[max(*i,hash(i)%1070)&(x*r)%5**10]for i in zip(c,*g)]for r in g]
#071	33d5333e	115	118	104	118	133	123	100	119	121	111	p=lambda g,a=0:[a*(v>0<r[-1+r])for i,v in enumerate(c)]if a or(c:=1-r[::-1].index(a:=v))]for r in g]
#072	3428a4f5	54	54	54	60	62	62	60	60	54	54	p=lambda g,u=1:(g*0!=0 and f*map(p,g,u*(7.7))or(g:=n)*3
#073	3618c87e	46	46	46	46	46	46	46	46	46	46	p=lambda g:3*g[:1]+g[3],[5,n+4 for n in g[2]]]
#074	3631a71a	77	77	102	81	80	89	99	82	112		p=lambda g,g[90]:or p(g*[zip(*map(map,[a][n]*30,p(g[:30]+g[:2]+g[::-1]])))
#075	363442ae	86	86	86	86	88	86	86	86	86	86	R=range(9):p=lambda g:[g[i]:i>4]+g[i-1%3+1][k-k/3+5]*g[1%3][k%3]for k in R]for i in R]
												def p(c): a=r for n in range(len(c)): for f in range(len(c[0])): if a[n][f]=1:=z:=f(n)} for n in r:=f+1,e+n)for l,n in z for f in range(-1,2)for e in range(-1,2)if e+n in range(len(c))!=f+1 in ran<b g(len(c[0]))!=0<r[e+n][f+1]] for n in(1,1,-1)*4: r=[z for*z,in zip(*r[:n])] for f in range(-13,13): if all(e+n in range(len(c))!=f+1 in range(len(c[0]))!=a[n][1]in(1,3,r[e+n][f+1])for l,n in z): for l,n in z:r[e+n][f+1]=a[n][1] return r
#076	36d67576	258	264	269	279	286	286	328	320	329	270	p=lambda i,k=r:*v:k and p([map(p,i,[k+1]*99,[*2]+i,[1:]+[1+2],*v]),k-1)or((c:=w.count)(2)+c(4)>=2:=1)*4or i
#078	3906ae3d	60	60	59	60	62	61	60	61	61	60	p=lambda i,k=r:*v:k and p([map(p,i,[k+1]*99,[*2]+i,[1:]+[1+2],*v]),k-1)or((c:=w.count)(2)+c(4)>=2:=1)*4or i
#079	39a864d4	107	121	105	123	123	121	123	123	123	123	p=eval(f'lambda a,max(b)=a {for*a,in map(zip,a,a[1:],a[2:])*2}if any(min(*a,*zip(*a))),key=b.count)")

#080	39a1d7f9	232	243	231	240	255	239	253	256	235	256	def p(f):n=f.index(min(f,key=seep))+1;e=f*u+1;for u,f in enumerate(f[:n]):return b
#081	3aa6b7a	90	81	83	96	95	100	95	96	95	91	n[[f or le-1u/n+1+o/n],*le(a+u-r)/n],*le(a+u-r)/n for n in e if e[r]=e[a]]<2;abs(c-r-1)]-1]]for o,f in enumerate(f[:n]):return b
#082	3ac3ab23	50	50	50	50	50	50	50	50	50	50	l in enumerate(f))for a in e if all(e.get(a+1)+u)for u,f in enumerate(f))l-1]]
#083	3af2558	40	40	40	40	40	40	40	40	40	40	p=1;lambda g:exec(f'g[:]=('q:=1)*g:=x.pop()or l]<[-1:q]for r in g[:]*7):"44)or g
#084	3bd67248	62	62	62	62	64	63	62	62	63	62	p=1;lambda g:[+r[:]-1]for r in g:g[:]-1]]
#085	3bb1bada	49	49	49	62	56	57	59	81	58	49	def p(g,i=0):
#086	3b6f4f3e	166	154	151	206	211	193	221	224	194	226	*h,i=g
#087	3e9b0459	36	36	36	36	36	36	36	36	36	36	for r in h:i-1;r[:]=2;l[:]=4
#088	3d62369	98	95	100	110	106	120	106	110	101	105	return g
#089	3e980e27	233	233	232	251	243	254	260	279	279	277	p=1;lambda g:g+0=0and g[:]=p(c)r[:r]eg for r in g]
#090	3eda0437	147	158	153	184	182	155	190	190	192	175	p=1;lambda i,k=r,*s=0:~k+1 or l[-(s= abs(a)or 1,s&8//4][y>0]-1>1) y,*k for x in sum(1,x)if 0<x1==y=0,0][k>6]fo+ b
#091	3f7978a0	62	62	62	69	63	90	64	63	64	63	p=1;lambda g:[r[:]-1]for r in g[:]-1]]
#092	4085323	86	87	86	94	100	92	95	101	99	101	exec(f'def p(g):t'g[:]=zip(c'g[any(h:=g[-1])-2:-1]):*48);return[lh[g<1]+*for g in g[:][1:-1]*2)
#093	4003f83a	98	98	98	108	118	117	102	100	113	100	def p(i):
#094	41a4f7e	102	96	97	124	102	131	111	115	101	130	for n in(e={u*1}+e:i for n,i in enumerate(i))for a,i in enumerate(i):if i):
#095	4285a5f9	70	70	74	79	77	73	97	79	83	85	for r in e:
#096	4290a0e	284	282	282	298	330	299	340	309	301	300	t=f
#097	42a50994	96	98	93	110	113	110	108	108	110	108	if e[n]==e[r]:
#098	4347f48a	74	72	64	88	96	98	96	99	97	106	for a in[e]*3:
#099	44480148	93	112	103	142	122	122	126	134	126	134	for i in o:b=f(-n)and t=-min(len(t+m(s-2))for s in range(5))&o for m in(1,20)*(min(u)-max(u))&o;n1=b;e=[le o+b
#100	445eab21	85	88	88	87	88	94	86	95	94	94	r a+m*20ln b and f=-min(len(t+m*(s-2))for s in range(5))&o for m in(1,20)*(min(u)-m)for m in r}}for a,e in enumerate b
#101	447f4412	271	265	258	275	285	297	290	326	340	307	return e
#102	4448ac46	126	124	135	161	183	186	150	148	174	194	i,r,o,u,n=[(a+m*2)for m,e in enumerate(e)for a,e in enumerate(e):if e==f)for f in range(5)]
#103	44f52b50	29	29	29	29	29	29	29	29	29	29	for f in o:l=f for f in o for m in r ln if abs(f-m)ln(1,20)}
#104	4522001f	84	84	84	89	96	88	89	94	94	89	for i in o:b=f(-n)and t=-min(len(t+m*(s-2))for s in range(5))&o for m in(1,20)*(min(u)-max(u))&o;n1=b;e=[le o+b
#105	4612ad53	134	124	139	161	175	153	155	126	152	214	r a+m*20ln b and f=-min(len(t+m*(s-2))for s in range(5))&o for m in(1,20)*(min(u)-m)for m in r}}for a,e in enumerate b
TASK #	TASK ID	TEAM #1	TEAM #2	TEAM #3	TEAM #4	TEAM #5	TEAM #6	TEAM #7	TEAM #8	TEAM #9	TEAM #10	p=1;lambda g,i=1,k=0,-1*g or l[-(c*(k=-b)(ln r))or(sum(map(bool,r))-b>2>1/6)*2)for c in r]for r in zip(*p,g,i-1):-b

#106	4644240e	67	59	55	66	67	74	66	67	66	66	p=lambda i,s=[];k=3;-k+1 or p[(zip(*i+s)).l[:~0].k-1] def p(t):z=1-on(4*str(t))-5;f=r+emge(s+z);return([i[k]/%2][y]/2)or(x-z+0+*1[0])[1]in[u:=y-z+0+*1[1][0],z*2+~u])%2d+c+b											
#107	469497ad	161	142	132	149	205	154	137	144	136	137	p=lambda i,s=0;a=0;0and(b)=fmap(p,a,[a].i=len(a)/2)))*b[:~0]or a%~aks r y in x]for x in x]											
#108	46f33f3e	52	52	66	46	59	61	91	62	46	64	57	63	p=lambda a,s=0;a=0;0and(b)=fmap(p,a,[a].i=len(a)/2)))*b[:~0]or a%~aks p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]									
#109	47c1f68c	80	80	77	41	81	91	91	90	82	90	82	90	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]									
#110	484b58aa	84	85	84	85	99	88	88	114	86	102	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#111	48481b45	60	60	60	61	62	70	61	60	61	60	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#112	483810c2	91	91	106	101	103	101	103	101	103	106	123	136	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]									
#113	4869b9ba	25	25	25	25	25	25	25	25	25	25	25	25	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]									
#114	4941d64f	64	64	64	65	65	65	65	65	65	65	65	65	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]									
#115	49e741c5	51	52	52	51	71	63	66	70	53	66	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#116	4c4377d9	20	20	20	20	20	20	20	20	20	20	20	20	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]									
#117	4c5c2c10	115	147	130	179	184	170	175	172	148	177	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#118	50846271	245	244	225	270	267	277	257	280	275	276	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#119	508b43b6	102	102	102	107	111	114	112	107	107	102	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#120	50cb28d2	89	85	78	106	99	108	105	99	100	106	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#121	5117e062	89	89	88	89	102	90	85	90	87	89	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#122	516b444c	70	75	71	89	90	81	80	78	89	89	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#123	53594a51	72	75	75	75	75	75	71	75	75	71	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#124	536b8214	97	96	95	103	108	109	98	110	125	105	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#125	543a8241	118	110	116	128	134	130	142	119	128	172	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#126	54482841	54	54	54	54	54	54	54	54	54	54	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#127	5449e175	57	63	63	59	65	72	79	75	79	75	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#128	5521cd49	57	59	59	59	61	61	59	59	61	64	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#129	55282efc	47	47	47	47	47	47	47	47	48	47	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#130	5614dbcf	61	65	65	65	65	65	65	65	65	65	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#131	56d12b01	126	117	125	160	127	128	147	132	131	128	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#132	56f19f63	86	86	86	104	89	104	106	126	117	101	p=lambda g:[fmap(max,*r+max(-a~-b for a,b in zip(r,t))+t for t in g))]for r in g]											
#133	57a9c2db	307	276	273	319	275	318	318	321	321	30												

#136	550a986e	94	104	97	10	110	112	110	107	110	110	110	def import re;p=lambda g:eval([g:re.sub('0?=(.34d)%s(2d)'%x,x.strip(g)[::-1])for x in re21*8] [-1])									
#137	5c2c9af4	139	137	141	146	157	142	146	143	146	146	146	def p(g):+,(a,b),b=sorted(zip(g,R:=range(len(g)))):C=map(a):return[C*(max(y-A,y-abs(x-A-x.index(C))%(4-b[1]))+b=0)for x in R]for x in R]									
#138	5b8aa888	103	103	104	119	120	124	119	116	115	125	128	p=lambda g,k:35:-k%g or p([g:=e[g][0]==g%e<49>k]for e in 1::-1])for l, in zip(*g[0]in g[0]:).k-1)									
#139	606b6152	92	89	88	97	102	82	101	100	102	103	103	p=lambda g:[a:0 or v%g[0]<(v[4]:).w:(4)](v%v[5]:).+7*or*v,w, in zip(*g,v)]for v in g]									
#140	616a0abd	36	36	36	36	36	36	36	36	36	36	36	p=lambda g:[l::-1]-l]for r in g[::-1]]									
#141	623a6a94	92	92	93	106	100	98	97	100	106	111	p=lambda g,l:=l:[map(max,r:(a:=abs(g.index(m:=max(g))+4:=1-1)))]+m,m[a]:+r,x]]for r in g]										
#142	65c24649	40	40	40	40	40	40	40	40	40	40	40	p=lambda g:[r+r[::-1]for r in g[::-1]]									
#143	63613498	127	135	130	142	155	145	151	212	139	169	def p(g,i:=1):										
#144	6430c8c4	53	53	53	59	59	60	59	59	53	53	53	+B,-G=b'%x'%g for k in b'XXXXXXXXXX'%BEH:R[R+1]=(48%[k]or-5)%k[R+1]+5 return(G)>+R)and eval(bytes(R))or p(g,1+1)									
#145	6455b5f1	177	167	157	197	227	196	229	187	220	225	p=lambda g,u:=l:[g:0]=and([map(p,g:u%g[5:]))or 3>g%u										
#146	6622240a	58	58	58	58	58	58	58	58	58	58	p=lambda g,k:7:-v:=1:-k%r or p([a:=b%k%or b//max(f:=sum(G,[]))+6m(n*(f%-(2))=b)+8,b%511*4,*[ak 2 b]*5,v:=v*512]*b										
#147	67385a82	81	81	72	89	86	92	89	87	87	113	k]for b in 2[+l]:[0:-1]for+*, in zip(*x),k-1)										
#148	673ef223	118	130	124	151	200	159	154	153	142	173	p=lambda g,X:=0:[s:=0,X:=any(r%k+X%6,[32%*(2+sum(r,-s)*(s:=e+v))or(X in g)*8for v in l,g:=g+8//8*[X]] [2]for r=b										
#149	6773b510	75	74	75	78	82	80	78	80	80	80	in g]										
#150	67a3c6ac	30	30	30	30	30	30	30	30	30	30	p=lambda g:[r[::]-1]for r in g]										
#151	67e4d23a	103	103	103	105	114	118	100	113	111	102	def p(g):										
#152	67e8384a	40	39	40	40	40	40	40	40	40	40	e,k=[-s.index(max(e))for s in(g,g[0])]										
#153	681b3aeb	118	133	131	173	150	137	128	162	139	119	for v in b'-(48)%x'%:lg[le+5%[k%v%3]=4										
#154	6855a6e4	98	98	100	119	112	122	103	107	109	116	T=0-1,2:p=lambda g:max(all(sum(G:=[g[k+1%g][y+1%g]_g[k-1%g][y-1%g-1]for y in T],G))%g for i in range(4b										
#155	68b16554	18	18	18	18	18	18	18	18	18	18	p=lambda g:[l::-1]										
#156	694412f3	118	142	128	161	177	160	159	181	172	180	import re;p=lambda g:eval(g:=re.sub('(?<4,{34}4) (?= {34}4(.+?.{3}4){?})',x"*(X:=g.count('X\X')//X('+'+1",s"#+										
#157	6a1e5592	240	238	242	247	239	288	249	247	247	281	def p(m):1,t,*x=15,sum(m,e:=[]),e:[e:=e+4n for n in range(150)[f[:1]:1]if(4>f)&[m]]>[]>[r:=x+[e,n]]for n in n										
#158	6aa20dc0	261	264	260	298	284	321	285	310	308	294	range(1+5*3):r[if+*e[n]]:e:[]]for f in range(150):]import re;p=lambda g:[zip(*([a/f]ln/e										
#159	6b9890af	103	107	105	115	107	114	141	141	112	187	def f, l=map(lambda g,*g:[zip(a:=[2]*99,*f for x in g or p(G,*g)for c in g[:1:3]if c.count(2)==2)f*(x)-[0,2]),a]										
#160	6c434455	104	105	100	118	106	138	116	119	109	136	import re;p=lambda i,*n:eval(sub('1,45)1,{25}77*3,x"0,2,0,1 2,2,2,0,2,0",str(n or p(1,+1))))										
#161	6cd42e23	82	82	79	83	89	82	83	87	81	80	p=lambda m:[(4//G:=sum(G,m).count(C)*x/4/(C(1)+4 for i in m[0])for k,x:= in m]										
#162	6cf79266	96	97	97	122	104	124	113	109	110	110	import re;p=lambda g,k:=0:eval('1,1,1',join(re.split('"(55)0,										
TASK #	TASK ID	TEAM #1	TEAM #2	TEAM #3	TEAM #4	TEAM #5	TEAM #6	TEAM #7	TEAM #8	TEAM #9	TEAM #10											

#163	6d016d9f	130	131	130	133	137	133	139	147	146	153	R=range(11);p=lambda g: [max(*g[y] [x]=55)or g[p/3+4+y//4] [p%-4 x//4]==4] [p%-4 x//4]==4] *g[p/3+4+y//4] [p%-4 x//4] for p in R) for c+b									
#164	6d0e6fbc	32	32	32	32	32	32	32	32	32	32	p=lambda g: [r+[:]-1] for r in g]									
#165	6d5a5a5d	112	130	130	122	127	133	120	127	142	149	p=lambda g: [zzip(*map(lambda x: r[:]- (k:=[:]-1), index(max(r, key=sum(g[:]-1, g), index())))+k*(max(r[:k:1]),)+r,*g2))]									
#166	6d7f5ebb	61	63	63	64	65	65	63	65	61	65	p=lambda g: [[:c>] [e]*99k]] *2for c,e in zip(*g,1)] for i in g]									
#167	6e02f1e7	72	71	70	76	83	82	77	72	75	76	p=lambda i: [[:5*(y=x*x)]*str(c)]/23 for x in b [x x]] for y in c(0,1,2)]									
#168	6e9193e8	108	110	110	112	127	116	135	116	116	111	import re;p=lambda g: eval (re.sub(r'0*(= -{35})', '(= -{70})', 42742, \2), r'\2', f'*(zip(*g[70:] for p in g(2)), j**))[:]-1]									
#169	6e82a1ae	108	110	111	130	127	137	129	133	132	135	p=lambda g,k=i,1,j=6,-k:=g or p[(a[:a-1]*a[:a]=c[h]*a[c] (1:=+8)) [k>0] for c in r[:]-1] for r,*in zip(*g),k-1,0)									
#170	6ecd11f4	166	163	163	170	215	174	191	180	200	193	f:=lambda t: [exoc"e"[:]] *zip(*[sum(c[0]) in t[0]] [:]-1);**82): i:p=lambda g: eval(f" {f'h'h'\n\n}for c, n in zip(r,n[h-b									
#171	6f68cd9b	51	51	51	51	53	56	61	51	53	51	p=lambda g: g[eval(g[0]) or p((*(zip(*g[:0:-1], [8]*9)))]									
#172	6fa7a4d4	20	20	20	20	20	20	20	20	20	20	p=lambda g: g[eval(g[:]-1)]									
#173	725221af	207	201	204	220	208	219	222	222	222	215	def p(t): x=[[:] in t+[:]/3] for t in range(9) for n in range(len(t)-2) if ([[:] in t+[:]/3] +b									
#174	72ca3f5d	91	89	97	98	102	110	96	105	99	105	1 in t+[:]/3] in t+[:]/3] for t in range(9))>1;return 1									
#175	73251a56	74	75	75	72	87	75	75	75	75	75	p=lambda g,i=i: eval(f'g={g:=[:]-1})*(g[:]=[:] for x in zip(*g) if {i} in x)),*(4[3] or p(g,i+1))									
#176	7447882a	64	64	61	72	67	70	76	68	75	75	p=lambda g: [g:=[]] or x for x,y,z in zip([0]+t+[:],r,c)) for c,r in zip(*g,g[2])									
#177	7468f01a	51	51	51	54	54	55	54	54	54	54	p=lambda g,x=5: [[:] *r(r:=x-x)]*6for x in s)* (r:=1)) for s in g]									
#178	74683b37	47	47	47	50	47	51	47	52	50	50	p=lambda g: g[:i+1] for p[2] for r in g if g[i]*=]									
#179	74dd1130	21	21	21	21	21	21	21	21	21	21	p=lambda g: [zzip(*g)]									
#180	75b8110e	79	79	74	82	83	84	82	84	83	82	p=lambda a: [p(b) for b in map(lambda a, a[4:]] or max(sum(a[4:])) [i];,key=pool)									
#181	760b3eac	67	67	67	67	69	74	67	67	67	67	def p(g): for i in g[:3]: x=g>g[3] [3] [1]: x:x+3]=15:2:-1] return g									
#182	776f6fc6	160	167	168	182	195	191	199	203	218	224	from re import*;p=lambda g,h=0: eval (sub(sub('2 3', '1', '(.*{4})', join(a=-findall('*,*5,{46}(.{15}) *5,s=-str*+b									
#183	777dfe62	79	93	94	93	94	93	94	93	98	93	(h or p(g,g[2])) [0]], "\Va", "\.join(a% (1,2,3,4), s))									
#184	780d0014	91	91	94	104	113	109	102	122	99	104	p=lambda g,h=0: g+=0and [map(r,1en(g)/2)*g[:1]+g[:1]*9,h or g] [2:-2] or h%7*g									
#185	783f9e64	107	110	121	143	126	112	149	120	128	158	p=lambda g,*p: [g:= [[:*(f:=f-rx):2]*max(Z)] for c,x in zip(g,r)] if (c*2) [1:1] for r in zip(*g[2]) ifor Z in g] [1]									
#186	79b02b6e	60	60	61	61	61	63	61	61	61	60	p=lambda g,i: [2,1]*k:=sum(sum(g, [0])) *2,2,h), [0,6,h,0],3*10]]									
#187	796d01b9	91	83	88	90	106	98	88	96	97	91	p=lambda g: [[:] *i:=c[2>-1]*c//for c in g]*[:]-1] for r,*in zip(*p(g,i-1))]									
#188	7b7f5f51	53	59	63	57	53	72	55	73	71	68	p=lambda g: (X:=g[:53]*-len(g[2])) *(g:=X-X) or [map(p,g)]									
#189	7d008303	95	111	111	111	113	116	113	111	109	110	p=lambda g,h=[[:] :g=0]-and [map(r,3*g[1]- (c'0 in "h% %g[2] *7] +3*g[1+1], (h%g[3>2])) or h%2*g									
#190	7dcd4fec	103	104	103	111	112	115	108	109</												

#200	8403a5bd5	84	84	84	84	88	89	91	111	91	93	115	def p(lambda g,c:=5): [g[9].index(m:=max(g[9]))*(0)+[m,c,m:c*(any(x)*g[9]))*(0)for x in g]]										
#201	846b1b03	187	198	188	188	230	215	248	246	229	231	224	def p(lambda f,i:=1):for r in zip(*g):if any(R:=[(x*(f:=f+1)or(x==4))or(g:=g+[x])*g]for x in r)):return [z:= [a,*[0]*2+e+b										
#202	885e0971	103	98	104	106	106	107	64	105	107	107	107	n(m),4),*[[a-1e[15]],*[[c:=a in m[0]or-1],g[-2]])for r*,in zip(*m),if any(c),z]										
#203	88c4e4ed	64	64	64	64	64	64	64	64	67	67	64	p=lambda g,g:=exec("v,g[:]=zip(*map(min,[a,v])&len(*v,*1,0)<3),v:=1)for i,j in g[::-1]]):,"*36)or g										
#204	8e8a6efa	93	93	93	93	99	106	119	113	128	102	105	import re;p=lambda g,g:=eval(re.sub(r'([1-1]).([1-1]+)',r'\1*(k:=len((2)))*[2+K/2+5]',1,strip(g)))										
#205	8731374e	144	134	136	180	153	190	218	179	181	208		n in g)<6]:[...:1],k-1)										
#206	88a10438	140	140	140	139	161	148	151	149	149	151		def p(a): i,j:=divmod(sum(a,[]).index(5),len(a[0])):a[a[j]]:=0 for s in eval("filter(any,zip(*+2+*a)))"):a[a[-1][-1:-j+2]]:=s,i+=1 return a										
#207	88a6d173	80	74	74	78	86	81	78	86	81	86	81	p=lambda g:[p(1)for*,i in map(zip,g,g[3:3])or min(q:=g[0]+g[1],key=q,count										
#208	890034e9	163	189	211	227	222	229	238	219	224	223		from re import*,p=lambda g:=eval('(:min(k:=str(g)+#[*],key=k.count)).join(split(sub(1,f')-[1])(' ,sub('"%s'+b										
#209	8a004b2b	278	268	247	250	273	287	314	304	296	277		def p(t): e=1 for o in e*:e=[*o]for o in zip(*e[4:])if(*o)-(-0,4)):i=[*o]for o in zip(*[4in 1(-1)-2:-:-1])] if[0]or v,f in enumerate(t):for a,f in enumerate(*o)for r,f in enumerate(*o)for i in [0,(6+20*[[]])][x:=t)//o+2+b										
#210	8be779e	20	20	20	20	20	20	20	20	20	20	20	0*(4))[(t-r)//o]for r,i in enumerate(t)for n,i in enumerate(t))*(o)for n,f in enumerate(*o)for i[1:r+1][a+n]in le["b										
#211	8a5021e8	48	48	48	48	48	48	48	48	48	48	48	p=lambda g:[r::-1]+r for r in g[[::-1]]*(g[*2]):[9]										
#212	8d510a79	99	101	93	111	113	99	120	117	121	121		p=lambda i,k:=18:"*1 or [(k.pop()or[0,*x])*(k-2)]%56-(5in x)for - in 1]for*,in zip(*p(1,k-1))]										
#213	8a181b1e	86	87	90	88	89	85	94	89	87	106		p=lambda g:[l[0:]for r in g if(l[1:=le for e in r if e[5])["o:=6-len(*"s"%g)"]):[l[0]										
#214	8a5b5113	86	87	90	88	89	85	94	89	87	106		p=lambda g:[r+g.pop()[(3):-1]for r,*r[4:]in zip(g+1,g[[::-1]])]										
#215	8a5b1be9a	42	42	42	42	42	42	42	42	42	42	42	p=lambda g:[*map(max,g,[3:6]+9,g[6:9]*9)]										
#216	8a5b1be9a	113	111	113	131	121	130	137	133	124	148		p=lambda g:=max([sorted(sum(v:=[(k[1:19]-1%3]for k in g[1%21:1%22]),[]),key=1...-eq,-)v]for i in range(4+*9))][1										
#217	8a2ea2aa	95	95	95	95	95	95	95	95	95	95	95	p=lambda g:[*1]iter(any(zip(*g[8:1]or p([c]d for c in a for d in b)for a in g[2]or b in g[1]))]										
#218	90c28c67	56	56	56	56	63	67	69	63	64	63	63	p=lambda g,*h:[*t:0]for x in zip(*h or p(*g))if any(*r)]										
#219	90f3ae37	242	241	242	248	266	303	302	241	280	320		def p(s): a=x[0],i:=3-any(x[2])>>any(*r[1]) while any(*[w]):a+=x[w],i+=1 for l in 0,1,2,-1: for w in range(16-len(a)): for y in range(2*(l<0),10): for f in range(1len(a)): for g in range(10): if 1+g in range(10) i=x[*+f][1]+g] i=a[g][g]:y*=g>y if 1+g in range(10) i=x[*+f][1]+g] i=0:y*=g*y for f in range(1len(a)): for g in range(10): if 1+g in range(10) i=x[*+f][1]+g] i=a[g][g]>0:y=x[*+f][1]+g] i=1 return x										
#220	913ab3ed	89	82	88	109	97	93	110	93	94	94	94	p=lambda g,i:=3-g:=1 or p([(f.pop()or[0,*z])[-1]]+*4%8/4,15)for r in g]for x in g[,1,-1)										
#221	91413438	86	87	86	89	98	98	87	100	90	80	80	def p(g):v:=sum(g,g).count(1:=0):return((r*(9+(1:=1-1)//3*v)+[0]*99))[:3+*g]for x in g[*]										
#222	91714a8b	102	107	93	117	109	110	137	119	108	141		p=lambda g:[g:=[(c*(str(*r*(g).count(2*(f:=c),-))>9)for c in r)for*,in zip(*g)]for - in g[5]										
#223	917f3fa0	46	51	46	46	53	54	50	52	50	50	p=lambda g,g:=1+*1-or g and p(g[0])]*3+*p(g[1:1])											
#224	928ad870	145	114	131	185	145	149	181	138	171	187	1	p=lambda g,i:=2:1-1+g or [g:=(0-+any(g[0])*i<6)],[[max(max(g))*99]]+[<4]+[+zip(*p([+zip(*g[0:1][:-:-1]),1,-1-1])][:-:-b										
TASK #	TASK ID	TEAM #1	TEAM #2	TEAM #3	TEAM #4	TEAM #5	TEAM #6	TEAM #7	TEAM #8	TEAM #9	TEAM #10												

[illegible]

[illegible]

TASK #	TASK ID	TEAM #1	TEAM #2	TEAM #3	TEAM #4	TEAM #5	TEAM #6	TEAM #7	TEAM #8	TEAM #9	TEAM #10	
#367	e7095f1d	135	128	126	130	129	146	126	129	127	223	import re;p=lambda g,k=-19:k*g or p(eval(re.sub(r'0(?=, 4.{(8-1)len(g)*3+4})5, 0,{(8-6))0},"a",str(zip(*g)))+b[::-1])),k+1)
#368	e7a8a8e	135	136	129	140	149	140	142	172	146	152	import re;p=lambda i:eval(re.sub("5, 5)*",lambda m:re.findall(r"[50](?:, [1-9])+",s*2)[-8]m.end()-1:-32).count(+b(5)),s:stx(4)))
#369	e839010	109	109	109	141	124	121	131	129	118	124	p=lambda g,n=r:g*n or lambda l:b=a+(a>4)or(a+b)/4-.25,min(a,b))[b>3]for a in r],p(g,n-1)[::-1])
#370	e8c4411	159	164	227	247	249	267	208	266	262	208	import re;p=lambda g:exec('s=f'+zip(g[::-1]),j",i:s.rfind(d=i("0")-i(j):min(s,key=1)):g[:]=eval(re.sub("(?=(, {c,b %d)+0)\d%d",j,s,d/8*d)-(len(g)*3+5)):','*4)or g
#371	e961658	102	109	109	109	124	125	109	125	118	109	p=lambda g,u=0:eval(G:=f'+zip(*u or p(g,g)),j"[: (u-g.rfind("1")>4.find("1")>1)-3]+*3,*3+G[r+5:1])
#372	e9a1cf9a	47	47	48	48	48	48	48	48	48	48	p=lambda g:eval(["map(max,g.pop(0),g[5]),","*6)
#373	e9a1cf9a	38	38	38	38	39	38	39	38	38	38	p=lambda g: [":=max(zip(*g))*3,T[::-1])
#374	ea32f347	103	106	107	148	106	133	124	116	122	111	p=lambda g,i=5:-1*g or p(eval(x:=f'+zip(*g)),)"_replace(1//5*",5",1//5*",7*"len((x))*5"))[::-1],1-1)
#375	ea786f4a	53	53	53	53	53	58	53	53	53	53	def p(g,i=0): for r in g:r[1]=r[1]=0;i+=1 return g
#376	eb281b96	30	30	30	30	30	30	30	30	30	30	p=lambda g,h=0:[h:=r:=for*x,in zip(*h)or p(g,g)]if r!=h]
#377	eb2a1d5d	55	55	55	60	59	61	60	59	59	60	import re;p=lambda g:exec("s=...*len(g)+.0'i:g[:=-1]=zip(*eval(re.sub(r'0(?={x},)*, ['0]{x*2}, (,)' ,r'\2'+b, str(g))),","*4)or g
#378	ec883f72	140	131	138	154	159	179	155	145	145	142	p=lambda g,i=47:(g*-1 or [(3&g)'(c:=r:=pop(0))%5]2kq-(8in x),"c*q[2+9,c&10][8>13-1//4]or c for q in [0]+r[:0:-1])f+b or*x,in zip(*p(g,i-1))
#379	ecdeab53	137	135	141	163	190	153	185	170	177	147	p=lambda g,i=47:(g*-1 or [(3&g)'(c:=r:=pop(0))%5]2kq-(8in x),"c*q[2+9,c&10][8>13-1//4]or c for q in [0]+r[:0:-1])f+b or*x,in zip(*p(g,i-1))
#380	ed56c47	27	27	27	27	27	27	27	27	27	27	p=lambda g: ["(a:=v,{f:=keys(sum(g,x)))][any(0<1.count(a[2])<4for i in[r,c])*2+0**v]for*c,v in zip(*g,x)]for+ b or*x,in zip(*p(g,i-1))
#381	ef13bb50	79	79	79	86	89	82	88	90	86	90	p=lambda g: ["(P:=r in g[::(9)or[P:=r:=pop(0)or any(r*p)*9for _ in g]for r in g]
#382	f15e1fac	133	129	124	138	146	123	167	179	131	152	p=lambda g,d=8,s=0:d and p(*zip(*g,["c,*[0]*(s:=s+c==2))*g[0][1:]]for c,*v in g][g[1>3]g[1]])[::-1]-(d/4>0-b),d-1)or g
#383	f1eetba8	123	120	121	130	128	136	121	129	138	124	p=lambda g:[[(a:=v,{f:=keys(sum(g,x)))][any(0<1.count(a[2])<4for i in[r,c])*2+0**v]for*c,v in zip(*g,x)]for+ b or*x,in g]
#384	f25fba4	62	61	62	67	69	65	63	68	66	65	p=lambda g,*a:sum([any(x)*2*[x]for*x,in zip(*a or p(*g)),1])
#385	f25fba3	25	25	25	25	25	25	25	25	25	25	p=lambda g:i:g[:4:-1]+g[5:]
#386	f2829549	50	50	52	52	53	54	52	52	52	50	p=lambda g: ["eval('3>r:=pop(0)+r[3],)*3]for r in g]
#387	f35d00a	192	201	205	230	222	235	231	215	252	235	p=lambda g:[(x[8&x-any(c)for*c,x in zip(*g,x)]*2for x in g]*2
#388	f5b8619d	61	61	62	62	62	62	62	61	61	61	p=lambda g,n=1:-n*g or [(1:=((x:=r:=pop(0)+>0)*-1)%2*sum(r[:2-1])*4/5*for x,x or-p**4/5*p*4,sum(*sum(g,x>9-b),[])-(6,*4,5)) or x][b//4]for p in [0]+r[:0:-1]]for*x,in zip(*p(g,n-1))if [1:=0]]
#389	f76d97a5	57	57	57	58	61	57	57	62	57	57	p=lambda g:[sum(*sum(g,x)-[5,x])for x in g]
#390	f8a81e49	98	98	100	121	113	122	103	107	109	116	import re;p=lambda g,k=0:eval(re.sub(r'c{2}(9)+2+*2+*2","*\[g<0] [:-1]",f'+zip(*k or p(g,g)),j""))
#391	f8b3a0a	63	63	63	63	63	63	63	63	63	63	p=lambda g:[zip(sorted(range(10),key=sum(g,g)).count)[7+4:-1]]
#392	f8c80d96	135	149	152	158	154	174	156	160	156		p=lambda g,n=23:-n*g or eval(f'r:=r:=pop(0)or[b,v:=max(max(g))[fn]//4n('0, %d, "%n*2in'({g}^-3))*any(r[-1:]),"*10)f+b or*x,in zip(*p(g,n-1))
#393	f8f40b80	63	63	63	63	63	63	63	63	63	63	p=lambda g:[zip(sorted(range(10),key=sum(g,g)).count)[8+5:-1])
#394	f9012d9	91	85	85	90	94	96	113	95	92	85	p=lambda g:[(1+1[1:])](-8161%len(g)-1,index(0))%6:][x]for i in g if(x=-1,count(0))
#395	fafa1fa7	53	53	53	58	56	58	56	59	53	53	p=lambda g,u=[]:g*0=[and(*map(p,g,u[g[3:]])or g**uk2
#396	fb5c5309	177	167	139	201	171	202	203	218	205	194	p=lambda m,n=266,f=0:[sum(*sum(m,[f]))for e in s]for r in m if(f:=(X:=m>5)*[a:=(r*3)[x:=n[32]]in(s:=r[x:=b x+X],[f]X))%a])or p(m,n-1)
#397	fc682909	117	110	113	126	145	130	124	132	135	124	p(lambda g,i=8):exec('1:=c=1//9-1,infcr _ in(a+1)(b:=%+%/s)*b)%s=3,3in"%((g[c:=c+1][1%9:1%9+2]),)*3)+1)or g
#398	fecae190	78	77	73	73	81	78	72	79	81	72	def p(g):g:=r:=g.count(0)/6+*0]:return[r:=1[1:]+[c]for c in g+r[5:]]
#399	ff8166a	63	62	63	64	66	66	64	64	64	64	p=lambda g:[zip(*[iter(sum(g,[]))%7*[1,0]+[0]*9])*(3)][3]
#400	ff80c23	67	67	66	67	70	67	67	67	67	67	p=lambda g:[b[:5]for r in g]if(b:=g.pop(0)[*r,1].index(:):-0))